ATS Friendly Resume

Aim

To design and develop a simple, ATS-friendly resume web page using semantic HTML and CSS, ensuring clean text extraction and a professional layout.

Algorithm

1. **Start**
2. **Initialize HTML Document:** Create a standard HTML5 skeleton, including the <!DOCTYPE html> declaration, <html>, <head>, and <body> tags.
3. **Configure Metadata:** Inside the <head> tag, set the character encoding to UTF-8, configure the viewport for responsive scaling, and set the document <title> to "Dhanush C K - Resume".
4. **Define Global Styles (CSS):** Open a <style> block to define visual rules: a. Apply a universal reset to remove default margins and padding. b. Set a background color for the body and define a central resume-container with a maximum width (e.g., 800px), white background, and professional standard fonts (like Arial or Segoe UI) for ATS readability.
5. **Structure the Header:** Inside the <body>, create a <header> element. a. Add an <h1> tag for the candidate's name (DHANUSH C K). b. Add a subtitle for the professional title (SOFTWARE DEVELOPER). c. Group contact links (Email, LinkedIn, GitHub, Website, Location) inside a designated flex container with visual separators.
6. **Structure the Main Content:** Create a <main> tag to hold the core resume data, divided into logical <section> elements.
7. **Add Summary Section:** a. Create an <h2> heading for "PROFESSIONAL SUMMARY". b. Add a paragraph detailing your frontend experience and logical problem-solving approach.
8. **Add Skills Section:** a. Create a section with an <h2> heading for "TECHNICAL SKILLS". b. Use a CSS grid layout to cleanly align categories (Programming, Web Technologies, Tools, Soft Skills) with their respective tool lists.
9. **Add Experience & Education Sections:** a. Create sections for "EXPERIENCE" and "EDUCATION". b. Use Flexbox layout (justify-content: space-between) to align job/degree titles to the left and dates to the right. c. List specific bullet points (<li>) for intern responsibilities and university affiliations.

10. **Add Projects Section:** a. Create a section with an <h2> heading for "PROJECTS". b. Use the Flexbox layout to align the project names. Describe the projects (Gradia Vision, Vijñana) and list their tech stacks clearly.
11. **Add Additional Details:** a. Create separate sections for "ACHIEVEMENTS", "LANGUAGES", and "ACTIVITIES & LEADERSHIP". b. Utilize unordered lists (<ul>) to cleanly outline awards, languages spoken, and campus roles.
12. **Finalize & Save:** Close all nested HTML tags properly (</section>, </main>, </body>, </html>), save the source file with an .html extension, and open it in a web browser to render the output.
13. **Stop**

Source Code

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Dhanush C K - Resume</title>

    <style>

        * {

    border: none;

    box-sizing: border-box;

    margin: 0;

}

body {

    font-family: 'Times New Roman', Times, serif;

    font-size: 1rem;

}

header {
```

```
    display: flex;
    align-items: center;
    flex-direction: column;
    margin: 1rem 0 0 0;
}
.name {
    font-size: 1.7rem;
    font-weight: 600;
}
.role {
    font-size: 1rem;
}
.place {
    font-size: 1rem;
}
.contact {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    text-align: center;
}
.bar {
    background: #000;
    width: 0.1rem;
    height: 1rem;
    margin: 0 0.5rem;
}
```

```
main {
    display: flex;
    align-items: center;
    flex-direction: column;
    margin: 0 0 2rem 0;
}
.horizontal-line {
    background-color: #000;
    height: 0.1rem;
    margin: 1rem 0;
    align-self: center;
}
.section {
    display: flex;
    flex-direction: column;
}
.horizontal-line, .section {
    width: min(80%, 900px);
}
.section-heading {
    font-size: 1.2rem;
    font-weight: 600;
    margin: 0 0 0.3rem 0;
}
.skills-grid {
    display: grid;
    grid-template-columns: auto auto 1fr;
```

```
    gap: 0.5rem;
}
.experience-grid {
    display: grid;
    grid-template-columns: auto 1fr auto;
    column-gap: 2rem;
    row-gap: 1rem;
}
.experience-role {
    font-size: 1.1rem;
}
.experience-content {
    display: flex;
    flex-direction: column;
}
.items {
    display: flex;
    flex-direction: column;
    margin: 0.3rem 0;
    gap: 0.3rem;
}
.items span::before {
    content: '- ';
}
.education-grid {
    display: grid;
    grid-template-columns: auto 1fr;
```

```
    row-gap: 1rem;
}
.education-details {
    display: flex;
    flex-direction: column;
    gap: 0.2rem;
}
.bold {
    font-weight: 600;
}
.year {
    justify-self: flex-end;
}
.project-grid {
    display: grid;
    grid-template-columns: 1fr;
    gap: 1rem;
}
.project-details {
    display: flex;
    flex-direction: column;
    row-gap: 0.3rem;
}
.project-details span:nth-child(n+2)::before {
    content: '- ';
}
.achievements-content {
```

```
    display: flex;
    flex-direction: column;
    gap: 0.3rem;
}
.achievements-content span::before {
    content: '- ';
}
.activities-content {
    display: flex;
    flex-direction: column;
    gap: 0.3rem;
}
.activities-content span:before {
    content: "- ";
}
@media (max-width: 600px) {
    .section,
    .horizontal-line {
        width: 94%;
    }
    .experience-grid {
        grid-template-columns: 1fr;
        row-gap: 0.4rem;
    }
    .experience-grid .year {
        justify-self: start;
    }
```

```
    .education-grid {
        grid-template-columns: 1fr;
        row-gap: 0.3rem;
    }
    .year {
        justify-self: start;
    }
    .skills-grid {
        grid-template-columns: 1fr;
        gap: 0.2rem;
    }
    .contact {
        row-gap: 0.3rem;
    }
    .bar {
        display: none;
    }
    .contact {
        flex-direction: column;
        gap: 0.2rem;
    }
}
@page {
    size: A4;
    margin: 0;
}
@media print {
```

```
    body {
        margin: 0;
    }
    main {
        margin-bottom: 0;
    }
    .section,
    .horizontal-line {
        width: 90%;
    }
}
</style>
</head>
<body>
    <header>
        <span class="name">DHANUSH C K</span>
        <span class="role">SOFTWARE DEVELOPER</span>

    <div class="contact">
            <a href="mailto:contact@dhanushck.in">contact@dhanushck.in</a>
            <span class="bar"></span>
            <a href="https://linkedin.com/in/dhanushck"
target="_blank">linkedin.com/in/dhanushck</a>
            <span class="bar"></span>
            <a href="https://github.com/dhanush-ck" target="_blank">github.com/dhanush-ck</a>
            <span class="bar"></span>
            <a href="https://dhanushck.in" target="_blank">dhanushck.in</a>
```

```
        <span class="bar"></span>
        <span class="place">Kannur, Kerala, India</span>
    </div>
</header>
<main>
    <div class="horizontal-line"></div>

    <section class="summary section">
        <span class="section-heading">PROFESSIONAL SUMMARY</span>
        <p>
```

Web Developer with hands-on experience in frontend development and familiarity with backend technologies. Skilled in building webpages, troubleshooting issues, and approaching problems with logical and practical thinking. I have actively contributed to my college's technical fest, Vijñana, where I also had the opportunity to develop its webpage. Passionate about continuous learning and expanding my skills across the web development field.

```
        </p>
    </section>
    <div class="horizontal-line"></div>
    <section class="skills section">
        <span class="section-heading">TECHNICAL SKILLS</span>
        <div class="skills-grid">
            <span>Programming</span>
            <span>-</span>
            <span>C, C++, Java, JavaScript, MySQL, Python</span>
            <span>Web Technologies</span>
            <span>-</span>
            <span>CSS, Django, HTML, React</span>
            <span>Tools & Technologies</span>
```

```
            <span>-</span>
            <span>Firebase, Figma, Git, GitHub, Linux, SEO, VS Code, WordPress</span>
            <span>Soft Skills</span>
            <span>-</span>
            <span>Adaptability, Debugging, Logical Thinking, Problem Solving,
Teamwork</span>
        </div>


    </section>
    <div class="horizontal-line"></div>
    <section class="experience section">
        <span class="section-heading">EXPERIENCE</span>
        <div class="experience-grid">
            <span>Feb 2026 - Mar 2026</span>
            <div class="experience-content">
                <span class="experience-role">Frontend Developer Intern - Cognifyz
Technologies</span>
                <div class="items">
                    <span>Improved proficiency in Tailwind CSS</span>
                    <span>Developed a responsive UI system</span>
                </div>
            </div>
            <span>Remote</span>
            <span>Dec 2024 - Dec 2024</span>
            <div class="experience-content">
                <span class="experience-role">Frontend Developer Intern -
CODEXINTERN</span>
                <div class="items">
```

```
            <span>Built website using react</span>
            <span>Learned React development practices</span>
          </div>
        </div>
        <span>Remote</span>
    </div>
</section>
<div class="horizontal-line"></div>
<section class="education section">
    <span class="section-heading">EDUCATION</span>
    <div class="education-grid">
        <div class="education-details">
            <span class="bold">Master of Computer Application</span>
            <span>College of Engineering, Trivandrum</span>
            <span>Affiliated to APJ Abdul Kalam Technological University</span>
        </div>
        <span class="year">2026 - Present</span>

        <div class="education-details">
            <span class="bold">Bachelor of Computer Application</span>
            <span>Kodiyeri Balakrishnan Smaraka Government College, Thalassery</span>
            <span>Affiliated to Kannur University</span>
        </div>
        <span class="year">2023 - 2026</span>
    </div>
</section>
<div class="horizontal-line"></div>
```

```
<section class="project section">
    <span class="section-heading">PROJECTS</span>
    <div class="project-grid">
        <div class="project-details">
            <span class="bold">Gradia Vision</span>
            <span>A smart academic platform that tracks student performance, predicts future scores, and provides personalized guidance through analytics and a mentor bot.</span>
            <span>CSS, Django, HTML, JavaScript, pdfplumber, Python, Scikit-learn</span>
        </div>
        <div class="project-details">
            <span class="bold">Vijñana Website</span>
            <span>A static website for college’s technical fest Vijñana (2025 & 2026) to manage registrations and provide event information.</span>
            <span>CSS, Firebase, HTML, JavaScript</span>
        </div>
    </div>
</section>
<div class="horizontal-line"></div>
<section class="achievements section">
    <span class="section-heading">ACHIEVEMENTS</span>
    <div class="achievements-content">
        <span>Graduated as Class Topper</span>
        <span>Secured 1st place in Fast Typing, Coding Competition, Blind Coding and Web Designing, and 2nd Place in Debugging Contest conducted by Kodiyeri Balakrishnan Smaraka Govt College Thalassery, January 2026</span>
        <span>Secured 1st place in the Cipher Up, debugging contest conducted by St. Thomas College of Engineering & Technology, April 2025</span>
```

Secured 1st Place in Decryption and Blind Coding, and 2nd Place in Fast Typing and Coding Competitions at Kodiyeri Balakrishnan Smaraka Govt College Thalassery, February 2025

</section>

<section class="languages section">

LANGUAGES

English, Hindi, Malayalam

</section>

<section class="activities section">

ACTIVITIES & LEADERSHIP

 Campus Lead, µLearn - Coordinated and led the technical fest team, actively contributing to the planning and execution of technical events.

Technical Fest Organizer - Conducted a debugging contest as part of the technical fest under the µLearn organization.

Actively participated in and contributed to various technical events conducted as part of the fest.

</section>

</main>

</body>

</html>

Output

DHANUSH C K

SOFTWARE DEVELOPER

contact@dhanushck.in | linkedin.com/in/dhanushck | github.com/dhanush-ck | dhanushck.in | Kannur, Kerala, India

PROFESSIONAL SUMMARY

Web Developer with hands-on experience in frontend development and familiarity with backend technologies. Skilled in building webpages, troubleshooting issues, and approaching problems with logical and practical thinking. I have actively contributed to my college's technical fest, Vijñana, where I also had the opportunity to develop its webpage. Passionate about continuous learning and expanding my skills across the web development field.

TECHNICAL SKILLS

Programming	- C, C++, Java, JavaScript, MySQL, Python
Web Technologies	- CSS, Django, HTML, React
Tools & Technologies	- Firebase, Figma, Git, GitHub, Linux, SEO, VS Code, WordPress
Soft Skills	- Adaptability, Debugging, Logical Thinking, Problem Solving, Teamwork

EXPERIENCE

Feb 2026 - Mar 2026 — Frontend Developer Intern - Cognifyz Technologies — Remote
- Improved proficiency in Tailwind CSS
- Developed a responsive UI system

Dec 2024 - Dec 2024 — Frontend Developer Intern - CODEXINTERN — Remote
- Built website using react
- Learned React development practices

EDUCATION

Master of Computer Application — 2026 - Present
College of Engineering, Trivandrum
Affiliated to APJ Abdul Kalam Technological University

Bachelor of Computer Application — 2023 - 2026
Kodiyeri Balakrishnan Smaraka Government College, Thalassery
Affiliated to Kannur University

PROJECTS

Gradia Vision
- A smart academic platform that tracks student performance, predicts future scores, and provides personalized guidance through analytics and a mentor bot.
- CSS, Django, HTML, JavaScript, pdfplumber, Python, Scikit-learn

Vijñana Website
- A static website for college's technical fest Vijñana (2025 & 2026) to manage registrations and provide event information.
- CSS, Firebase, HTML, JavaScript

ACHIEVEMENTS

- Graduated as Class Topper
- Secured 1st place in Fast Typing, Coding Competition, Blind Coding and Web Designing, and 2nd Place in Debugging Contest conducted by Kodiyeri Balakrishnan Smaraka Govt College Thalassery, January 2026
- Secured 1st place in the Cipher Up, debugging contest conducted by St. Thomas College of Engineering & Technology, April 2025
- Secured 1st Place in Decryption and Blind Coding, and 2nd Place in Fast Typing and Coding Competitions at Kodiyeri Balakrishnan Smaraka Govt College Thalassery, February 2025

LANGUAGES

English, Hindi, Malayalam

ACTIVITIES & LEADERSHIP

- **Campus Lead, µLearn** - Coordinated and led the technical fest team, actively contributing to the planning and execution of technical events.
- **Technical Fest Organizer** - Conducted a debugging contest as part of the technical fest under the µLearn organization.
- Actively participated in and contributed to various technical events conducted as part of the fest.

Result

The HTML and CSS code for the ATS-friendly resume was successfully written, executed, and verified in a web browser. The resulting layout efficiently uses semantic tags (like <header>, <main>, and <section>) to ensure that automated resume parsers can accurately read the candidate's data.

Internal Links

Aim

Design and develop a webpage on the topic “Web Technology” using HTML and CSS. The webpage should demonstrate the use of internal links through a navigation menu. Each link should navigate to the corresponding section within the same webpage.

Algorithm

1. Initialize the HTML document with standard DOCTYPE, head, and body tags.
2. Configure character encoding, viewport settings, document title, and external resource links inside the head element.
3. Define CSS styles including color variables, responsive grid layouts, header styling, and keyframe animations.
4. Build a header containing the main webpage title and a responsive hamburger menu icon.
5. Construct a sticky navigation bar with internal links pointing to corresponding section anchors.
6. Create content sections for Client and Server components using styled cards and icon lists.
7. Develop an interactive tabbed interface displaying View, Business, and Data Access web application layers.
8. Assemble a grid section displaying various web content types using styled chip tags.
9. Outline HTTP protocols, request methods, and response structures.
10. Detail characteristics of dynamic web pages and render an application server architecture flow diagram.
11. List web security fundamentals using highlighted security cards.
12. Add a footer section containing author copyright details.

Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Web Technology</title>
    <link rel="stylesheet"href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-icons.min.css">
    <style>
:root {
    --primary: #18181b;
    --primary-light: #71717a;
    --primary-dark: #09090b;
    --bg-main: #f4f4f5;
    --bg-card: #ffffff;
    --text-main: #18181b;
    --text-muted: #334155;
    --chip-bg: #e4e4e7;
    --chip-text: #27272a;
    --accent: #f59e0b;
}
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: system-ui, -apple-system, sans-serif;
```

```
}
body {
    background-color: var(--bg-main);
    color: var(--text-main);
    line-height: 1.6;
}
header {
    background: var(--primary);
    color: white;
    position: sticky;
    top: 0;
    z-index: 1000;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}
.header-top {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 1rem 2rem;
}
header h1 {
    font-size: 1.8rem;
    margin: 0;
    text-align: center;
    flex-grow: 1;
}
.hamburger {
```

```
    display: none;
    background: none;
    border: none;
    color: white;
    font-size: 2rem;
    cursor: pointer;
    transition: transform 0.2s;
}
.header-spacer {
    width: 32px;
}
nav {
    background: var(--primary-dark);
    display: flex;
    justify-content: center;
    gap: 1.5rem;
    flex-wrap: wrap;
    padding: 0.75rem 2rem;
}
nav a {
    color: rgba(255, 255, 255, 0.85);
    text-decoration: none;
    font-size: 0.95rem;
    font-weight: 500;
    transition: color 0.2s;
}
nav a:hover {
```

```
    color: white;
}
main {
    max-width: 1200px;
    margin: 3rem auto;
    padding: 0 1.5rem;
}
.section {
    margin-bottom: 5rem;
}
.section-header {
    font-size: 2rem;
    color: var(--primary);
    border-bottom: 3px solid var(--primary-light);
    padding-bottom: 0.5rem;
    margin-bottom: 2rem;
}
.section-sub-header {
    font-size: 1.3rem;
    margin: 1.5rem 0 1rem;
    color: var(--primary-dark);
}
p {
    margin-bottom: 1rem;
    color: var(--text-muted);
}
.card {
```

```
    background: var(--bg-card);
    border-radius: 12px;
    padding: 2rem;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03);
    border: 1px solid rgba(113, 113, 122, 0.2);
}
.client-server-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 2rem;
}
.card-header {
    display: flex;
    align-items: center;
    gap: 1rem;
    margin-bottom: 1.5rem;
    color: var(--primary);
}
.client-server-logo {
    font-size: 2.2rem;
}
.styled-list {
    margin-left: 1.5rem;
    color: var(--text-muted);
}
.styled-list li {
    margin-bottom: 0.5rem;
```

```
}
.dynamic-tabs-container {
    background: var(--bg-card);
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
    border: 1px solid rgba(113, 113, 122, 0.2);
}
.tab-buttons {
    display: flex;
    background: #f8fafc;
    border-bottom: 1px solid #e2e8f0;
}
.tab-btn {
    flex: 1;
    padding: 1.2rem 1rem;
    border: none;
    background: none;
    font-size: 1rem;
    font-weight: 600;
    color: var(--text-muted);
    cursor: pointer;
    transition: all 0.2s ease;
}
.tab-btn:hover {
    background: var(--chip-bg);
    color: var(--primary-dark);
```

```
}
.tab-btn.active {
    background: var(--bg-card);
    color: var(--primary);
    border-bottom: 3px solid var(--primary);
}
.tab-content {
    padding: 2.5rem;
}
.item {
    display: none;
    animation: fadeIn 0.4s ease forwards;
}
.item.active-layer {
    display: block;
}
@keyframes fadeIn {
    from { opacity: 0; transform: translateY(10px); }
    to { opacity: 1; transform: translateY(0); }
}
.item h3 {
    font-size: 1.5rem;
    color: var(--primary);
    margin-bottom: 1rem;
}
.web-content-grid {
    display: grid;
```

```
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 1.5rem;
}
.item-list {
    display: flex;
    flex-wrap: wrap;
    gap: 0.6rem;
    margin-top: 1.5rem;
}
.item-list span {
    background: var(--chip-bg);
    color: var(--chip-text);
    padding: 0.35rem 0.85rem;
    border-radius: 99px;
    font-size: 0.85rem;
    font-weight: 600;
    border: 1px solid var(--primary-light);
    transition: background 0.2s;
}
.item-list span:hover {
    background: #d4d4d8;
}
.item-list span.highlight-chip {
    background: #fef3c7;
    border-color: var(--accent);
    color: #92400e;
}
```

```
.architecture {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 1.5rem;
    flex-wrap: wrap;
    background: var(--bg-card);
    padding: 3rem;
    border-radius: 12px;
    margin-top: 2rem;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
    border: 1px solid rgba(113, 113, 122, 0.2);
}
.arch-box {
    background: var(--primary);
    color: white;
    padding: 1rem 1.5rem;
    border-radius: 8px;
    font-weight: bold;
}
.architecture i {
    color: var(--primary-dark);
    font-size: 2rem;
}
.security-list {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

```
    gap: 1.5rem;
}
.security-card {
    border-left: 5px solid var(--primary);
    padding: 1.5rem;
}
footer {
    background: var(--text-main);
    text-align: center;
    padding: 1rem;
    margin-top: 5rem;
}
footer p {
    color: white;
}
@media (max-width: 768px) {
    .client-server-container {
        grid-template-columns: 1fr;
    }

    .tab-buttons {
        flex-direction: column;
    }
    .architecture {
        flex-direction: column;
    }
    .architecture i {
```

```
        transform: rotate(90deg);
    }

    .hamburger {
        display: block;
    }
    nav {
        display: none;
        flex-direction: column;
        width: 100%;
        padding: 1rem 0;
    }
    nav.active {
        display: flex;
    }
    nav a {
        padding: 0.8rem 2rem;
        width: 100%;
        text-align: center;
        border-bottom: 1px solid rgba(255, 255, 255, 0.1);
    }
    nav a:last-child {
        border-bottom: none;
    }
}
</style>
</head>
```

```
<body>
    <header>
        <div class="header-top">
            <button id="hamburger-btn" class="hamburger" aria-label="Toggle Menu">
                <i class="bi bi-list"></i>
            </button>
            <h1>Web Technology</h1>
            <div class="header-spacer"></div>
        </div>
        <nav id="nav-menu">
            <a href="#client-server">Client/Server</a>
            <a href="#web-application">Web Application</a>
            <a href="#web-content">Web Content</a>
            <a href="#http">HTTP</a>
            <a href="#dynamic-pages">Dynamic Pages</a>
            <a href="#application-server">Application Servers</a>
            <a href="#web-security">Web Security</a>
        </nav>
    </header>
    <main>
        <section id="client-server" class="section">
            <h2 class="section-header">Client &amp;amp; Server</h2>
            <div class="client-server-container">
                <article class="client card">
                    <div class="card-header">
                        <i class="bi bi-people-fill client-server-logo"></i>
                        <h3 class="client-server-heading">Client</h3>
```

```
        </div>
        <p>
            A client is a device or application that requests services or resources from a server over a network. When a user enters a website address in a browser, the browser acts as a client and sends a request to the web server.
        </p>
        <ul class="styled-list">
            <li>Web browsers such as Chrome, Firefox and Edge</li>
            <li>Mobile applications</li>
            <li>Desktop applications</li>
        </ul>
    </article>
    <article class="server card">
        <div class="card-header">
            <i class="bi bi-hdd-stack-fill client-server-logo"></i>
            <h3 class="client-server-heading">Server</h3>
        </div>
        <p>
            A server is a computer or software system that provides services, data, or resources to clients over a network. A web server receives requests from clients and sends the requested web pages or other resources as responses.
        </p>
        <ul class="styled-list">
            <li>Receives requests from clients</li>
            <li>Processes the requested information</li>
            <li>Sends the requested resources as a response</li>
        </ul>
    </article>
```

```
</section>
<section id="web-application" class="section">
  <h2 class="section-header">Components of Web Application</h2>
  <div class="dynamic-tabs-container">
    <div class="tab-buttons">
      <button class="tab-btn active" data-target="view-layer">View Layer</button>
      <button class="tab-btn" data-target="business-layer">Business Layer</button>
      <button class="tab-btn" data-target="data-layer">Data Access Layer</button>
    </div>
    <div id="layer-items" class="tab-content">
      <article id="view-layer" class="item active-layer">
        <h3>View Layer</h3>
        <p>The View Layer is the part of an application that the user sees and interacts with. It is responsible for displaying information and collecting user input.</p>
        <p><strong>Technologies:</strong> HTML, CSS and JavaScript.</p>
        <p><strong>Examples:</strong> Forms, buttons, menus and webpage layouts.</p>
      </article>
      <article id="business-layer" class="item">
        <h3>Business Layer</h3>
        <p>The Business Layer contains the application's logic and rules. It processes user requests and determines what should happen based on the application's requirements.</p>
        <p><strong>Example:</strong> Checking whether a username and password are correct during login.</p>
      </article>
      <article id="data-layer" class="item">
        <h3>Data Access Layer</h3>
        <p>The Data Access Layer is responsible for communicating with databases or other data storage systems. It retrieves, adds, updates, or deletes data.</p>
```

```
                <p><strong>Example:</strong> Retrieving a user's profile information from a database.</p>
            </article>
        </div>
    </div>
</section>
<section id="web-content" class="section">
    <h2 class="section-header">Types of Web Content</h2>
    <div class="web-content-grid">
        <article class="grid-item card">
            <h3 class="item-header">Text</h3>
            <p>Text is the most basic type of web content.</p>
            <div class="item-list">
                <span>Headings</span><span>Paragraphs</span><span>Articles</span><span>Menus</span><span>Links</span>
            </div>
        </article>
        <article class="grid-item card">
            <h3 class="item-header">Images</h3>
            <p>Images make webpages more attractive and help communicate information visually.</p>
            <div class="item-list">
                <span>JPEG/JPG</span><span>PNG</span><span>GIF</span><span>WebP</span><span>Logos</span><span>Icons</span>
            </div>
        </article>
        <article class="grid-item card">
            <h3 class="item-header">Audio</h3>
```

```
            <p>Audio allows websites to provide sound or music.</p>
            <div class="item-list">
                <span>Music</span><span>Podcasts</span><span>Voice recordings</span><span>Sound effects</span>
            </div>
        </article>
        <article class="grid-item card">
            <h3 class="item-header">Video</h3>
            <p>Video combines moving images and sound to present information.</p>
            <div class="item-list">
                <span>Movies</span><span>Tutorials</span><span>Advertisements</span><span>Educational videos</span>
            </div>
        </article>
        <article class="grid-item card">
            <h3 class="item-header">Animation</h3>
            <p>Animation involves creating movement or visual changes on a webpage.</p>
            <div class="item-list">
                <span>Loading animations</span><span>Moving objects</span><span>Animated menus</span><span>CSS transitions</span>
            </div>
        </article>
    </div>
</section>
<section id="http" class="section">
    <h2 class="section-header">HTTP and HTTP Request</h2>
    <article>
        <h3 class="section-sub-header">Definition of HTTP</h3>
```

HTTP stands for HyperText Transfer Protocol. It is a communication protocol used to transfer resources between a client and a web server.

</article>

<article id="request-response">

<h3 class="section-sub-header">HTTP Request and Response</h3>

An HTTP request is a message sent by the client to the server asking for a resource or requesting an action.

GET - Request data

POST - Send data

PUT - Update data

DELETE - Delete data

QUERY - Search/Query data

An HTTP response is the message sent by the server back to the client. It contains:

Status code

Headers

Requested content

</article>

</section>

<section id="dynamic-pages" class="section">

<h2 class="section-header">Dynamic Web Pages</h2>

A dynamic web page is a webpage whose content can change based on user interaction, data, time, or other conditions.

<h3 class="section-sub-header">Characteristics</h3>

```
        <div class="list item-list">
            <span>Changes based on input</span>
            <span>Retrieves from databases</span>
            <span>Personalized content</span>
            <span>Responds to interactions</span>
        </div>
    </section>
    <section id="application-server" class="section">
        <h2 class="section-header">Application Servers</h2>
        <p>An application server is software that provides an environment for running application logic and processing client requests.</p>
        <h3 class="section-sub-header">Simple Architecture</h3>
        <div class="architecture">
            <div class="arch-box">Client</div>
            <i class="bi bi-arrow-right"></i>
            <div class="arch-box">Web Server</div>
            <i class="bi bi-arrow-right"></i>
            <div class="arch-box">Application Server</div>
            <i class="bi bi-arrow-right"></i>
            <div class="arch-box">Database</div>
        </div>
    </section>
    <section id="web-security" class="section">
        <h2 class="section-header">Web Security</h2>
        <div class="security-list">
            <div class="card security-card">
                <strong>HTTPS</strong>
```

```
                <p>Encrypts communication between the browser and the server.</p>
            </div>
            <div class="card security-card">
                <strong>Strong Passwords</strong>
                <p>Strong and unique passwords help prevent unauthorized access.</p>
            </div>
            <div class="card security-card">
                <strong>Input Validation</strong>
                <p>Checked before processing to prevent malicious data from being submitted.</p>
            </div>
            <div class="card security-card">
                <strong>Authentication & Authorization</strong>
                <p>Verifies identity and determines what an authenticated user can access.</p>
            </div>
        </div>
    </section>
</main>
<footer>
    <p>&copy; 2026 Dhanush C K</p>
</footer>
<script>
document.addEventListener("DOMContentLoaded", () => {
    const tabButtons = document.querySelectorAll('.tab-btn');
    const tabLayers = document.querySelectorAll('.item');
    tabButtons.forEach(button => {
        button.addEventListener('click', () => {
            tabButtons.forEach(btn => btn.classList.remove('active'));
```

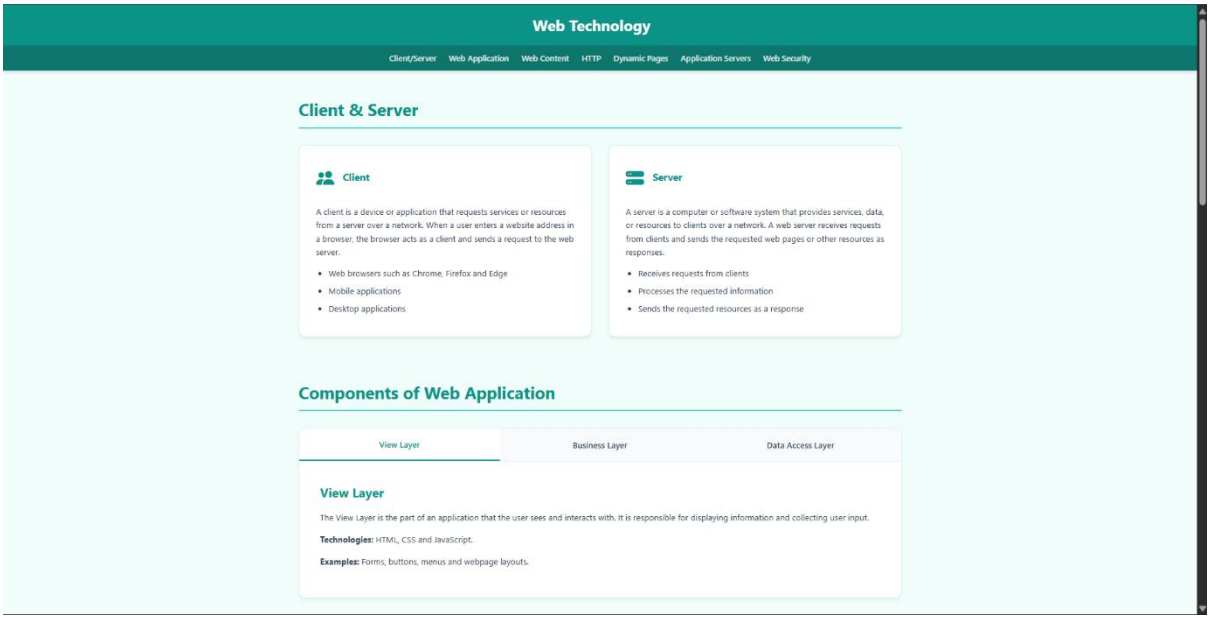
```
        tabLayers.forEach(layer => layer.classList.remove('active-layer'));
        button.classList.add('active');

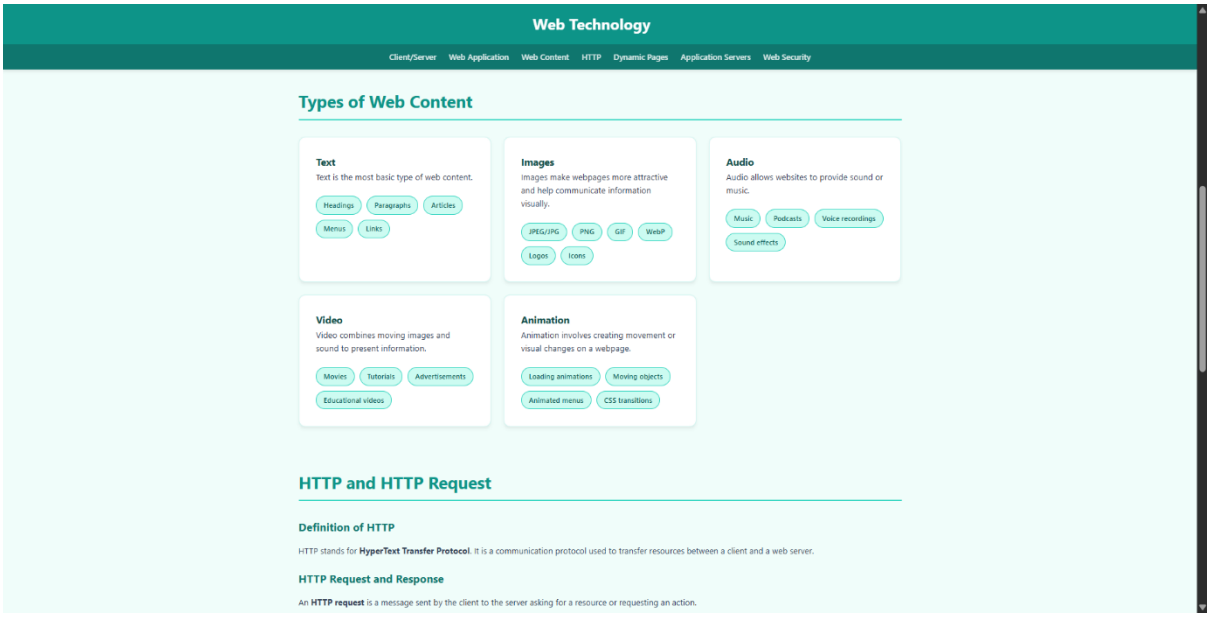
        const targetId = button.getAttribute('data-target');
        const targetLayer = document.getElementById(targetId);
        if(targetLayer) {
            targetLayer.classList.add('active-layer');
        }
    });
});
const hamburgerBtn = document.getElementById('hamburger-btn');
const navMenu = document.getElementById('nav-menu');
const menuIcon = hamburgerBtn.querySelector('i');
hamburgerBtn.addEventListener('click', () => {
    navMenu.classList.toggle('active');

    if (navMenu.classList.contains('active')) {
        menuIcon.classList.remove('bi-list');
        menuIcon.classList.add('bi-x-lg');
    } else {
        menuIcon.classList.remove('bi-x-lg');
        menuIcon.classList.add('bi-list');
    }
});
const navLinks = navMenu.querySelectorAll('a');
navLinks.forEach(link => {
    link.addEventListener('click', () => {
```

```
            navMenu.classList.remove('active');
            menuIcon.classList.remove('bi-x-lg');
            menuIcon.classList.add('bi-list');
        });
    });
});
    </script>
</body>
</html>
```

Output





Result

The webpage for “Web Technology” was successfully designed and developed using HTML and CSS. It incorporates a functional, responsive navigation bar providing seamless internal linking across distinct content sections: Client & Server architecture, Components of Web Application (featuring interactive View, Business, and Data Access layers), Types of Web Content, HTTP protocols, Dynamic Web Pages, Application Server flow, and Web Security fundamentals. The final output accurately reflects the structured layout and visual styling as shown in the preview.

Frames

Aim

Design a webpage with frames using HTML and CSS. The webpage should contain a menu frame and a content frame, where links from the menu frame open the corresponding webpages within the content frame.

Algorithm

1. Create a container document designed to partition the viewport into multiple functional regions without standard body page flow.
2. Set the title to "Frameset Layout".
3. Split the window vertically into top (20% height) and bottom (80% height) rows.
4. Subdivide the bottom region into left (25% width) and right (75% width) columns.
5. Set the top frame source to header.html.
6. Set the left frame source to menu.html.
7. Set the right frame source to home.html as the default landing view.
8. Include a noframes fallback block for non-supporting user agents.
9. Initialize the header document with centralized text and background color #0f766e.
10. Add an h1 tag displaying the department name and a subtitle describing the context.
11. Build a structured navigation menu with modular block buttons.
12. Assign target="contentFrame" to menu links to display documents in the content frame.
13. Create the Home page with an introductory heading and overview description.
14. Create the About page describing program objectives.
15. Create the Courses page displaying course codes, titles, and credit hours in a table.
16. Create the Contact page with styled accent borders showing email and location.
17. Optionally replace frameset with HTML5 Flexbox layout and responsive iframes.
18. Save all linked HTML documents within the same relative root directory.
19. Open index.html in a web browser to verify dynamic frame navigation.

Source Code

index.html

```
<!DOCTYPE html>
<html lang="en">
  <link>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Frames Webpage</title>
    <link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><rect width='100' height='100' rx='20' fill='%231c1c1c'/><text x='50' y='70' text-anchor='middle' font-family='Arial' font-size='65' font-weight='bold' fill='white'>F</text></svg>"></link>
    <style>
      * {
        box-sizing: border-box;
        margin: 0;
        padding: 0;
      }
      body {
        width: 100%;
        height: 100vh;
      }
      .container {
        display: flex;
        width: 100%;
        height: 100vh;
      }
      .menu-frame {
```

```
            width: 20%;
            height: 100%;
            border: none;
            min-width: 0;
        }
        .content-frame {
            width: 80%;
            height: 100%;
            border: none;
            min-width: 0;
        }
    </style>
</head>
<body>
    <div class="container">
        <iframe src="./menu.html" class="menu-frame" name="menu"></iframe>
        <iframe src="./home.html" class="content-frame" name="content"></iframe>
    </div>
</body>
</html>
```

example.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Examples</title>
```

```
<style>
  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }
  body {
    min-height: 100vh;
    background: #111;
    color: #fff;
    font-family: Arial, sans-serif;
    padding: clamp(20px, 5vw, 50px);
  }
  .tag {
    display: inline-block;
    padding: 6px 12px;
    margin-bottom: 15px;
    background: #333;
    border-radius: 20px;
    color: #aaa;
    font-size: 12px;
    letter-spacing: 1px;
  }
  h1 {
    font-size: clamp(28px, 5vw, 45px);
    margin-bottom: 12px;
  }
```

```
.intro {
    max-width: 750px;
    color: #aaa;
    line-height: 1.7;
    font-size: 16px;
    margin-bottom: 30px;
}
.examples {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 18px;
}
.card {
    padding: 25px;
    background: #1c1c1c;
    border: 1px solid #333;
    border-radius: 15px;
}
.number {
    display: inline-flex;
    width: 32px;
    height: 32px;
    align-items: center;
    justify-content: center;
    margin-bottom: 15px;
    background: #333;
    border-radius: 50%;
```

```
    color: #fff;
    font-size: 14px;
}
h2 {
    font-size: 20px;
    margin-bottom: 10px;
}
.card p {
    color: #aaa;
    line-height: 1.6;
    font-size: 14px;
    margin-bottom: 15px;
}
pre {
    padding: 15px;
    background: #0a0a0a;
    border: 1px solid #292929;
    border-radius: 8px;
    overflow-x: auto;
    color: #ccc;
    font-family: Consolas, monospace;
    font-size: 13px;
    line-height: 1.6;
}
.full {
    grid-column: 1 / -1;
}
```

```
        .result {
            margin-top: 15px;
            padding: 12px;
            background: #242424;
            border-radius: 8px;
            color: #aaa;
            font-size: 13px;
        }
        @media (max-width: 700px) {
            .examples {
                grid-template-columns: 1fr;
            }
            .full {
                grid-column: auto;
            }
        }
    </style>
</head>
<body>
    <span class="tag">PRACTICAL EXAMPLES</span>
    <h1>Frame Examples</h1>
    <p class="intro">
        The following examples demonstrate how frames and iframes
        can be used to divide a webpage and display different
        HTML documents.
    </p>
    <div class="examples">
```

```
        <div class="card">
            <span class="number">1</span>
            <h2>Old Frame Layout</h2>
            <p>
                The traditional method used a frameset to divide
                the browser window into multiple frames.
            </p>
            <pre>&lt;frameset cols="20%,80%"&gt;
    &lt;frame src="menu.html"&gt;
    &lt;frame src="home.html"&gt;
&lt;/frameset&gt;</pre>
            <div class="result">
                Divides the window into 20% and 80% sections.
            </div>
        </div>
        <div class="card">
            <span class="number">2</span>
            <h2>Modern Iframe</h2>
            <p>
                An iframe can embed another HTML document inside
                a normal webpage.
            </p>
            <pre>&lt;iframe
    src="home.html"
    name="content"&gt;
&lt;/iframe&gt;</pre>
            <div class="result">
```

```
            Displays home.html inside the iframe.
        </div>
    </div>
    <div class="card">
        <span class="number">3</span>
        <h2>Target Attribute</h2>
        <p>
            The target attribute specifies where the linked
            webpage should be displayed.
        </p>
        <pre>&lt;a href="about.html"
  target="content"&gt;
   About Frames
&lt;/a&gt;</pre>
        <div class="result">
            Opens about.html inside the content iframe.
        </div>
    </div>
    <div class="card">
        <span class="number">4</span>
        <h2>20 : 80 Layout</h2>
        <p>
            CSS can be used to create the required menu and
            content layout using percentage widths.
        </p>
        <pre>.menu-frame {
   width: 20%;
```

```
}
.content-frame {
   width: 80%;
}</pre>
          <div class="result">
             Menu occupies 20% and content occupies 80%.
          </div>
       </div>
       <div class="card full">
          <span class="number">5</span>
          <h2>Complete Navigation</h2>
          <p>
             Combining an iframe with the target attribute
             allows a navigation menu to control the content
             displayed in another frame.
          </p>
          <pre>&lt;iframe
   src="menu.html"
   class="menu-frame"&gt;
&lt;/iframe&gt;
&lt;iframe
   src="home.html"
   name="content"
   class="content-frame"&gt;
&lt;/iframe&gt;</pre>
          <div class="result">
             The menu remains on the left while the selected
```

```
            webpage opens in the 80% content area.
        </div>
    </div>
</div>
</body>
</html>
```

frame.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Frames</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }
        body {
            min-height: 100vh;
            background: #111;
            color: #fff;
            font-family: Arial, sans-serif;
            padding: clamp(20px, 5vw, 50px);
        }
        .tag {
```

```
    display: inline-block;
    padding: 6px 12px;
    margin-bottom: 15px;
    background: #333;
    border-radius: 20px;
    color: #aaa;
    font-size: 12px;
    letter-spacing: 1px;
}
h1 {
    font-size: clamp(28px, 5vw, 45px);
    margin-bottom: 12px;
}
.intro {
    max-width: 800px;
    color: #aaa;
    line-height: 1.7;
    font-size: 16px;
    margin-bottom: 30px;
}
.section {
    margin-bottom: 25px;
    padding: 25px;
    background: #1c1c1c;
    border: 1px solid #333;
    border-radius: 15px;
}
```

```
.section h2 {
    font-size: 22px;
    margin-bottom: 12px;
}
.section p {
    color: #aaa;
    line-height: 1.6;
    font-size: 14px;
    margin-bottom: 15px;
}
.old {
    border-left: 4px solid #777;
}
.new {
    border-left: 4px solid #aaa;
}
.label {
    display: inline-block;
    padding: 5px 10px;
    margin-bottom: 12px;
    background: #333;
    border-radius: 5px;
    font-size: 12px;
    color: #ccc;
}
pre {
    padding: 18px;
```

```
    background: #0a0a0a;
    border-radius: 8px;
    overflow-x: auto;
    color: #ccc;
    font-family: Consolas, monospace;
    font-size: 13px;
    line-height: 1.6;
}
.comparison {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}
.comparison th,
.comparison td {
    padding: 12px;
    border: 1px solid #333;
    text-align: left;
    font-size: 13px;
}
.comparison th {
    background: #292929;
    color: white;
}
.comparison td {
    color: #aaa;
}
```

```
        .note {
            margin-top: 25px;
            padding: 18px;
            background: #1c1c1c;
            border-radius: 10px;
            border: 1px solid #333;
            color: #aaa;
            line-height: 1.6;
            font-size: 14px;
        }
        @media (max-width: 600px) {
            .section {
                padding: 18px;
            }
            .comparison {
                font-size: 12px;
            }
            .comparison th,
            .comparison td {
                padding: 8px;
            }
        }
    </style>
</head>
<body>
    <span class="tag">HTML FRAMES</span>
    <h1>About Frames</h1>
```

```
    <p class="intro">
        Frames allow different HTML documents to be displayed in
        separate sections of a webpage. HTML has traditionally
        provided frames using the &lt;frame&gt; element, while modern
        webpages use the &lt;iframe&gt; element for embedding content.
    </p>
    <section class="section old">
        <span class="label">OLD METHOD</span>
        <h2>&lt;frame&gt; and &lt;frameset&gt;</h2>
        <p>
            In older versions of HTML, the &lt;frameset&gt; element
            was used to divide the browser window into multiple
            sections. Each section was created using a
            &lt;frame&gt; element.
        </p>
        <p>
            A separate HTML document could be loaded into each frame.
            The size of the frames could be specified using rows or
            columns.
        </p>
        <pre>
&lt;frameset cols="20%,80%"&gt;
    &lt;frame src="menu.html"&gt;
    &lt;frame src="home.html" name="content"&gt;
&lt;/frameset&gt;</pre>
        <p style="margin-top: 15px;">
            The menu frame could then use the target attribute to
```

```
        open pages in the content frame.
      </p>
      <pre>&lt;a href="about.html" target="content"&gt;
   About Frames
&lt;/a&gt;
      </pre>
   </section>
   <section class="section new">
      <span class="label">MODERN METHOD</span>
      <h2>&lt;iframe&gt;</h2>
      <p>
         The &lt;iframe&gt; element is the modern way to embed
         another HTML document inside a webpage. Unlike the old
         &lt;frame&gt; element, an iframe can be placed inside the
         normal HTML &lt;body&gt;.
      </p>
      <p>
         CSS can be used to control its size, position, layout,
         and responsiveness.
      </p>
      <pre>&lt;div class="container"&gt;
   &lt;iframe src="menu.html"
         class="menu-frame"&gt;
   &lt;/iframe&gt;
   &lt;iframe src="home.html"
         name="content"
         class="content-frame"&gt;
```

```
    &lt;/iframe&gt;
&lt;/div&gt;</pre>
        <p style="margin-top: 15px;">
            A link can load its webpage inside the iframe using
            the target attribute.
        </p>
        <pre>&lt;a href="about.html"
   target="content"&gt;
    About Frames
&lt;/a&gt;</pre>
    </section>
    <section class="section">
        <h2>Old vs Modern</h2>
        <table class="comparison">
            <tr>
                <th>Feature</th>
                <th>&lt;frame&gt;</th>
                <th>&lt;iframe&gt;</th>
            </tr>
            <tr>
                <td>HTML element</td>
                <td>&lt;frame&gt;</td>
                <td>&lt;iframe&gt;</td>
            </tr>
            <tr>
                <td>Used with</td>
                <td>&lt;frameset&gt;</td>
```

```
            <td>Normal HTML body</td>
        </tr>
        <tr>
            <td>HTML5</td>
            <td>Obsolete</td>
            <td>Supported</td>
        </tr>
        <tr>
            <td>CSS styling</td>
            <td>Limited</td>
            <td>Flexible</td>
        </tr>
        <tr>
            <td>Responsive design</td>
            <td>Difficult</td>
            <td>Easy with CSS</td>
        </tr>
    </table>
</section>
<div class="note">
    <strong>In this project:</strong>
    The modern &lt;iframe&gt; element is used to create the
    menu and content areas. The menu occupies 20% of the page,
    while the content area occupies 80%. The
    <strong>target</strong> attribute loads the selected page
    inside the content iframe.
</div>
```

```
</body>
</html>
```

home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home Page</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }
        body {
            min-height: 100vh;
            background: #111;
            color: #fff;
            font-family: Arial, sans-serif;
            padding: clamp(20px, 5vw, 60px);
        }
        .hero {
            padding: clamp(30px, 5vw, 55px);
            background: linear-gradient(135deg, #1e1e1e, #292929);
            border: 1px solid #3a3a3a;
            border-radius: 20px;
```

```
    margin-bottom: 30px;
}
.tag {
    display: inline-block;
    padding: 6px 12px;
    margin-bottom: 15px;
    background: #333;
    border-radius: 20px;
    color: #aaa;
    font-size: 12px;
    letter-spacing: 1px;
}
h1 {
    font-size: clamp(30px, 5vw, 52px);
    margin-bottom: 15px;
}
.hero p {
    max-width: 750px;
    color: #bbb;
    font-size: clamp(15px, 2vw, 18px);
    line-height: 1.7;
}
.section-title {
    margin-bottom: 18px;
    font-size: clamp(20px, 3vw, 28px);
}
.cards {
```

```
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 18px;
}
.card-link {
    text-decoration: none;
    color: inherit;
}
.card {
    height: 100%;
    padding: 25px;
    background: #1c1c1c;
    border: 1px solid #333;
    border-radius: 16px;
    transition: 0.2s ease;
}
.card-link:hover .card {
    transform: translateY(-5px);
    background: #242424;
    border-color: #555;
}
.icon {
    font-size: 30px;
    margin-bottom: 15px;
}
.card h3 {
    margin-bottom: 10px;
```

```
            font-size: 19px;
        }
        .card p {
            color: #999;
            line-height: 1.6;
            font-size: 14px;
        }
        .note {
            margin-top: 25px;
            padding: 18px;
            border-left: 3px solid #777;
            background: #1a1a1a;
            color: #aaa;
            line-height: 1.6;
            font-size: 14px;
        }
        @media (max-width: 600px) {
            .cards {
                grid-template-columns: 1fr;
            }
        }
    </style>
</head>
<body>
    <section class="hero">
        <span class="tag">HTML & CSS</span>
        <h1>Understanding HTML Frames</h1>
```

```
    <p>
        Welcome to our website dedicated to HTML frames.
        Learn how webpages can be divided into sections and
        how navigation can be used to display different pages
        within a content area.
    </p>
</section>
<h2 class="section-title">Explore the Website</h2>
<div class="cards">
    <a href="frame.html" target="content" class="card-link">
        <div class="card">
            <div class="icon">▣</div>
            <h3>About Frames</h3>
            <p>
                Learn what HTML frames are, how they work, and
                understand concepts such as frames, iframes,
                and the target attribute.
            </p>
        </div>
    </a>
    <a href="examples.html" target="content" class="card-link">
        <div class="card">
            <div class="icon">◇</div>
            <h3>Examples</h3>
            <p>
                Explore practical examples of frame layouts,
                iframe usage, navigation, and displaying
```

```
                different webpages within a content frame.
            </p>
        </div>
    </a>
</div>
<div class="note">
    <strong>Tip:</strong>
    Use the navigation menu or the cards above to switch
    between the different pages. The selected page will be
    displayed in the content area.
</div>
</body>
</html>
```

menu.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Menu Page</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }
        body {
```

```
    min-height: 100vh;
    background: #111;
    color: #fff;
    font-family: Arial, sans-serif;
    padding: 25px 15px;
    container-type: inline-size;
}
.header {
    text-align: center;
    margin-bottom: 25px;
}
.tag {
    display: inline-block;
    padding: 6px 10px;
    margin-bottom: 10px;
    background: #333;
    border-radius: 20px;
    color: #aaa;
    font-size: 10px;
    letter-spacing: 1px;
}
h2 {
    font-size: clamp(18px, 2vw, 22px);
}
.menu {
    display: flex;
    flex-direction: column;
```

```
    gap: 12px;
}
.menu a {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 14px;
    color: #ccc;
    background: #1c1c1c;
    border: 1px solid #333;
    border-radius: 12px;
    text-decoration: none;
    font-size: clamp(12px, 1.2vw, 15px);
    transition: 0.2s ease;
}
.icon {
    display: flex;
    align-items: center;
    justify-content: center;
    height: 32px;

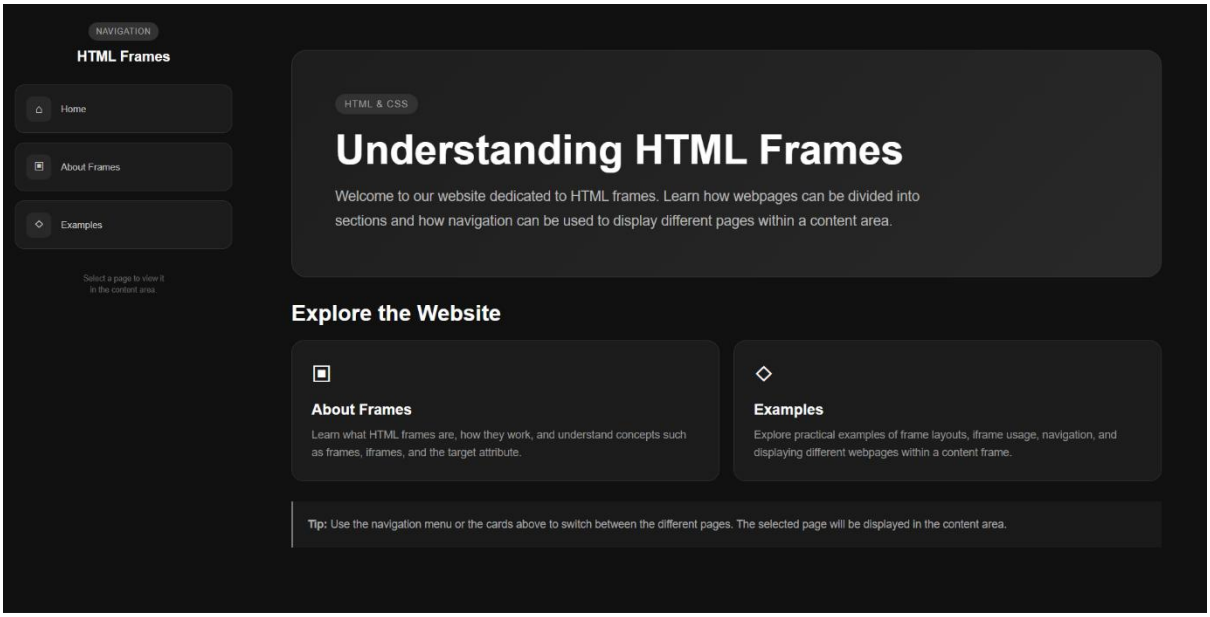
    background: #292929;
    border-radius: 8px;
    font-size: 15px;
}
.menu a:hover {
    background: #242424;
```

```
    border-color: #555;
    color: #fff;
    transform: translateX(3px);
}
.menu a:hover .icon {
    background: #333;
}
.menu a:active {
    transform: translateX(1px);
}
.footer {
    margin-top: 30px;
    text-align: center;
    color: #666;
    font-size: 10px;
    line-height: 1.5;
}
@container (max-width: 180px) {
    .header h2,
    .tag,
    .menu a span:not(.icon),
    .footer {
        display: none;
    }
    .header {
        margin-bottom: 20px;
    }
```

```
            .menu {
                gap: 10px;
            }
            .menu a {
                justify-content: center;
                padding: 10px;
            }
            .icon {
                min-width: 36px;
                height: 36px;
                font-size: 17px;
            }
        }
    </style>
</head>
<body>
    <header class="header">
        <span class="tag">NAVIGATION</span>
        <h2>HTML Frames</h2>
    </header>
    <nav class="menu">
        <a href="home.html" target="content">
            <span class="icon">⌂</span>
            <span>Home</span>
        </a>
        <a href="frame.html" target="content">
            <span class="icon">▣</span>
```

```
            <span>About Frames</span>
        </a>
        <a href="examples.html" target="content">
            <span class="icon">◇</span>
            <span>Examples</span>
        </a>
    </nav>
    <div class="footer">
        Select a page to view it<br>
        in the content area.
    </div>
</body>
</html>
```

Output



Result

The HTML and CSS webpage with frames/iframes and dynamic menu navigation was successfully created, tested, and verified in the web browser. As shown in the output preview, selecting links from the left navigation frame correctly loads and renders the corresponding webpages within the designated right content frame, maintaining the responsive 20:80 split layout as intended.

Job Application Form

Aim

Design and develop a Job Application Form using HTML and CSS. The webpage should provide an organized form for collecting the applicant's personal, contact, and professional information.

Algorithm

1. **Start.**
    - Define the HTML5 document type.
    - Open the core HTML tags.
    - Set basic metadata and title.
    - Import Google Fonts.
2. **Configure CSS styling:**
    - Apply universal CSS resets.
    - Define root CSS variables.
    - Style the page body background.
    - Style containers with glassmorphism.
    - Define responsive grids and field states.
    - Add responsive media queries.
3. **Build Form Structure:**
    - Create the header section.
    - Add Personal Information fields.
    - Add Security Information fields.
    - Add Contact Information fields.
    - Add Language selection checkboxes.
    - Add CV file upload input.
    - Add footer buttons and notice.
4. **Implement JavaScript Validation:**
    - Select form elements by ID.

- Create UI helper functions.
- Write field validation routines.

5. **Attach Event Listeners:**
   - Listen for real-time input events.
   - Listen for file change events.
   - Handle form submission validation.
   - Handle form reset events.
6. **Finalize:**
   - Close all HTML tags.
7. **Stop.**

Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Job Application Form</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Ubuntu:wght@300;400;500;600;700&display=s
wap" rel="stylesheet">
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }
```

```
:root {
    --primary: #7c3aed;
    --primary-dark: #6d28d9;
    --primary-light: #a78bfa;
    --text: #18181b;
    --muted: #71717a;
    --glass: rgba(255, 255, 255, 0.42);
    --glass-strong: rgba(255, 255, 255, 0.58);
    --border: rgba(255, 255, 255, 0.65);
    --error: #dc2626;
    --radius: 20px;
}
body {
    min-height: 100vh;
    font-family: "Ubuntu", sans-serif;
    color: var(--text);
    background:
        radial-gradient(circle at 10% 10%,
            rgba(167, 139, 250, 0.55),
            transparent 30%),
        radial-gradient(circle at 90% 20%,
            rgba(196, 181, 253, 0.6),
            transparent 28%),
        radial-gradient(circle at 70% 90%,
            rgba(139, 92, 246, 0.4),
            transparent 30%),
        linear-gradient(135deg,
```

```
            #f5f3ff,
            #ede9fe,
            #f8fafc);
    background-attachment: fixed;
}
.page {
    width: 100%;
    min-height: 100vh;
    padding: 60px 20px;
}
.form {
    width: 100%;
    max-width: 780px;
    margin: auto;
}
.form-header {
    position: relative;
    display: flex;
    align-items: center;
    gap: 22px;
    padding: 34px;
    background: var(--glass-strong);
    backdrop-filter: blur(25px);
    -webkit-backdrop-filter: blur(25px);
    border: 1px solid var(--border);
    border-radius: 24px;
    box-shadow:
```

```
        0 20px 60px rgba(76, 29, 149, 0.12),
        inset 0 1px 0 rgba(255, 255, 255, 0.7);
    margin-bottom: 20px;
    overflow: hidden;
}
.form-header::before {
    content: "";
    position: absolute;
    width: 180px;
    height: 180px;
    top: -100px;
    right: -50px;
    background: rgba(255, 255, 255, 0.35);
    border-radius: 50%;
    filter: blur(5px);
    pointer-events: none;
}
.header-icon {
    width: 64px;
    height: 64px;

    flex-shrink: 0;
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    background:
```

```
        linear-gradient(135deg,
            #8b5cf6,
            #6d28d9);
    border: 1px solid rgba(255, 255, 255, 0.5);

    border-radius: 17px;
    box-shadow:
        0 10px 25px rgba(109, 40, 217, 0.25);
}
.eyebrow {
    font-size: 11px;
    font-weight: 700;
    letter-spacing: 1.7px;

    color: var(--primary);
    margin-bottom: 7px;
}
.form-header h1 {
    font-size: 34px;
    font-weight: 600;
    margin-bottom: 7px;
}
.description {
    color: var(--muted);
    font-size: 15px;
}
.section {
```

```
    position: relative;
    background: var(--glass);

    backdrop-filter: blur(22px);
    -webkit-backdrop-filter: blur(22px);
    border: 1px solid var(--border);

    border-radius: var(--radius);
    padding: 32px;
    margin-bottom: 18px;
    box-shadow:
      0 15px 40px rgba(76, 29, 149, 0.08),
      inset 0 1px 0 rgba(255, 255, 255, 0.65);
    overflow: hidden;
}
.section::before {
    content: "";
    position: absolute;
    width: 150px;
    height: 150px;

    top: -100px;
    right: -60px;
    border-radius: 50%;
    background: rgba(255, 255, 255, 0.25);

    filter: blur(10px);
```

```
    pointer-events: none;
}
.section-heading {
    position: relative;
    display: flex;
    align-items: center;
    gap: 15px;
    margin-bottom: 30px;
}
.section-number {
    width: 42px;
    height: 42px;
    flex-shrink: 0;
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--primary);
    background: rgba(255, 255, 255, 0.38);

    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.7);
    border-radius: 12px;
    font-size: 13px;
    font-weight: 700;
}
.section-heading h2 {
    font-size: 20px;
```

```
    font-weight: 600;
    margin-bottom: 4px;
}
.section-heading p {
    color: var(--muted);
    font-size: 13px;
}
.fields {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 25px 20px;
}
.field {
    display: flex;
    flex-direction: column;
}
.field.full {
    grid-column: 1 / -1;
}
.field label {
    font-size: 14px;
    font-weight: 500;
    margin-bottom: 8px;
}
.field label span {
    color: var(--error);
}
```

```
.field input {
    width: 100%;
    padding: 13px 15px;
    font-family: inherit;
    font-size: 14px;

    color: var(--text);

    background: rgba(255, 255, 255, 0.35);
    border: 1px solid rgba(255, 255, 255, 0.65);
    border-radius: 11px;
    outline: none;
    transition:
        border-color 0.2s,
        background 0.2s,
        box-shadow 0.2s;
}
.field input:hover {
    background: rgba(255, 255, 255, 0.5);
    border-color: rgba(124, 58, 237, 0.3);
}
.field input:focus {
    background: rgba(255, 255, 255, 0.65);
    border-color: var(--primary);
    box-shadow:
        0 0 0 3px rgba(124, 58, 237, 0.1);
}
```

```
.field input::placeholder {
    color: #8b8b95;
}
.field input.input-error {
    border-color: var(--error);
    box-shadow:
        0 0 0 3px rgba(220, 38, 38, 0.08);
}
.field input.input-success {
    border-color: #16a34a;
}
.hint {
    margin-top: 6px;
    color: var(--muted);
    font-size: 11px;
}
.error {
    min-height: 17px;
    margin-top: 6px;
    color: var(--error);
    font-size: 12px;
}
.language-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px;
}
```

```
.language-option {
    position: relative;
    display: flex;
    align-items: center;

    gap: 12px;
    padding: 15px;
    background: rgba(255, 255, 255, 0.25);
    border: 1px solid rgba(255, 255, 255, 0.65);
    border-radius: 13px;
    cursor: pointer;
    transition: 0.2s;
}
.language-option:hover {
    background: rgba(255, 255, 255, 0.45);
    transform: translateY(-1px);
}
.language-option input {
    position: absolute;
    opacity: 0;
}
.checkmark {
    width: 20px;
    height: 20px;
    flex-shrink: 0;
    display: flex;
    align-items: center;
```

```
    justify-content: center;
    border: 2px solid #c4b5fd;

    border-radius: 6px;
    color: white;
    font-size: 11px;
    transition: 0.2s;
}
.language-option input:checked+.checkmark {
    background: var(--primary);
    border-color: var(--primary);
}
.language-option strong {
    display: block;
    font-size: 14px;
}
.language-option small {
    display: block;
    margin-top: 3px;
    color: var(--muted);
    font-size: 11px;
}
.upload-box {
    display: flex;
    align-items: center;
    gap: 16px;
    padding: 22px;
```

```
    background: rgba(255, 255, 255, 0.25);
    border: 1.5px dashed rgba(124, 58, 237, 0.4);
    border-radius: 14px;
    cursor: pointer;
    transition: 0.2s;
}
.upload-box:hover {
    background: rgba(255, 255, 255, 0.45);
    border-color: var(--primary);
}
.upload-icon {
    width: 44px;
    height: 44px;
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    background:
        linear-gradient(135deg,
            #8b5cf6,
            #6d28d9);
    border-radius: 11px;
    font-size: 25px;
}
.upload-box strong {
    font-size: 14px;
}
```

```
.upload-box p {
    color: var(--muted);
    font-size: 12px;
    margin-top: 4px;
}
.upload-box input {
    display: none;
}
.form-footer {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-top: 25px;
}
.form-footer button {
    border: none;
    border-radius: 11px;
    padding: 13px 22px;
    font-family: inherit;
    font-size: 14px;
    font-weight: 500;
    cursor: pointer;
    transition: 0.2s;
}
.reset-btn {
    color: var(--muted);
    background: rgba(255, 255, 255, 0.3);
```

```
    border: 1px solid rgba(255, 255, 255, 0.6) !important;
}
.reset-btn:hover {
    background: rgba(255, 255, 255, 0.6);
    color: var(--text);
}
.submit-btn {
    display: flex;
    align-items: center;
    gap: 15px;
    color: white;
    background:
        linear-gradient(135deg,
            #8b5cf6,
            #6d28d9);
    box-shadow:
        0 10px 25px rgba(109, 40, 217, 0.25);
}
.submit-btn:hover {
    transform: translateY(-2px);
    box-shadow:
        0 14px 30px rgba(109, 40, 217, 0.32);
}
.submit-btn span {
    font-size: 18px;
}
.footer-note {
```

```
    text-align: center;
    margin-top: 18px;
    color: var(--muted);
    font-size: 11px;
}
@media (max-width: 650px) {
    .page {
        padding: 25px 12px;
    }
    .form-header {
        padding: 25px 20px;
        border-radius: 18px;
    }
    .header-icon {
        width: 52px;
        height: 52px;
        border-radius: 14px;
    }
    .form-header h1 {
        font-size: 26px;
    }
    .description {
        font-size: 13px;
    }
    .section {
        padding: 24px 20px;
    }
```

```
        .fields {
            grid-template-columns: 1fr;
            gap: 20px;
        .field.full {
            grid-column: auto;
        }
        .language-grid {
            grid-template-columns: 1fr;
        }
        .form-footer {
            margin-top: 20px;
        }
    }
    @media (prefers-reduced-motion: reduce) {
        *,
        *::before,
        *::after {
            transition: none !important;
        }
    }
  </style>
</head>
<body>
  <main class="page">
    <form class="form" id="jobForm" novalidate>
      <div class="form-header">
        <div class="header-icon">
```

```
            <svg width="30" height="30" viewBox="0 0 24 24" fill="none"
xmlns="http://www.w3.org/2000/svg"
                aria-hidden="true">
                <rect x="3" y="7" width="18" height="13" rx="2" stroke="currentColor" stroke-
width="1.8" />
                <path d="M8 7V5.5C8 4.67 8.67 4 9.5 4H14.5C15.33 4 16 4.67 16 5.5V7"
stroke="currentColor"
                    stroke-width="1.8" stroke-linecap="round" />
                <path d="M3 12H21" stroke="currentColor" stroke-width="1.8" />
                <path d="M10 12V14H14V12" stroke="currentColor" stroke-width="1.8" stroke-
linejoin="round" />
            </svg>
        </div>
        <div>
            <p class="eyebrow">CAREER OPPORTUNITY</p>
            <h1>Job Application</h1>
            <p class="description">
                Tell us a little about yourself and your experience.
            </p>
        </div>
    </div>
    <section class="section">
        <div class="section-heading">
            <div class="section-number">01</div>
            <div>
                <h2>Personal Information</h2>
                <p>Basic information about you</p>
            </div>
```

```
            </div>
            <div class="fields">
                <div class="field">
                    <label for="name">Full Name <span>*</span></label>
                    <input type="text" id="name" placeholder="John Doe" required>
                    <small class="error" id="nameError"></small>
                </div>
                <div class="field">
                    <label for="age">Age <span>*</span></label>
                    <input type="number" id="age" placeholder="21" min="15" max="100"
required>
                    <small class="error" id="ageError"></small>
                </div>
            </div>
        </section>
        <section class="section">
            <div class="section-heading">
                <div class="section-number">02</div>
                <div>
                    <h2>Security Information</h2>
                    <p>Create your account credentials</p>
                </div>
            </div>
            <div class="fields">
                <div class="field">
                    <label for="password">Password <span>*</span></label>
                    <input type="password" id="password" placeholder="Enter a strong password"
required>
```

```
                <small class="error" id="passwordError"></small>
            </div>
            <div class="field">
                <label for="confirmPassword">
                    Confirm Password <span>*</span>
                </label>
                <input type="password" id="confirmPassword" placeholder="Retype your password" required>
                <small class="error" id="confirmPasswordError"></small>
            </div>
            <div class="field">
                <label for="securityQuestion">
                    Security Question <span>*</span>
                </label>
                <input type="text" id="securityQuestion" placeholder="What is your favourite color?" required>
                <small class="error" id="securityQuestionError"></small>
            </div>
            <div class="field">
                <label for="securityAnswer">
                    Security Answer <span>*</span>
                </label>
                <input type="text" id="securityAnswer" placeholder="Your answer" required>
                <small class="error" id="securityAnswerError"></small>
            </div>
        </div>
    </section>
    <section class="section">
```

```
            <div class="section-heading">
                <div class="section-number">03</div>
                <div>
                    <h2>Contact Information</h2>
                    <p>How can we reach you?</p>
                </div>
            </div>
            <div class="fields">
                <div class="field full">
                    <label for="email">Email Address <span>*</span></label>
                    <input type="email" id="email" placeholder="john@example.com" required>
                    <small class="error" id="emailError"></small>
                </div>
                <div class="field">
                    <label for="phone">Phone Number <span>*</span></label>
                    <input type="tel" id="phone" placeholder="9876543210" maxlength="10"
required>
                    <small class="hint">
                        Enter a 10-digit mobile number
                    </small>
                    <small class="error" id="phoneError"></small>
                </div>
                <div class="field">
                    <label for="homePhone">
                        Home Phone
                    </label>
                    <input type="tel" id="homePhone" placeholder="Optional" maxlength="10">
```

```
            <small class="error" id="homePhoneError"></small>
        </div>
    </div>
</section>
<section class="section">
    <div class="section-heading">
        <div class="section-number">04</div>
        <div>
            <h2>Languages</h2>
            <p>Select the languages you know</p>
        </div>
    </div>
    <div class="language-grid">
        <label class="language-option">
            <input type="checkbox" name="languages" value="English">
            <span class="checkmark">✓</span>
            <span>
                <strong>English</strong>
                <small>English</small>
            </span>
        </label>
        <label class="language-option">
            <input type="checkbox" name="languages" value="Hindi">
            <span class="checkmark">✓</span>
            <span>
                <strong>Hindi</strong>
                <small>हिन्दी</small>
```

```
            </span>
        </label>
        <label class="language-option">
            <input type="checkbox" name="languages" value="Malayalam">
            <span class="checkmark">✓</span>
            <span>
                <strong>Malayalam</strong>
                <small>മലയാളം</small>
            </span>
        </label>
    </div>
</section>
<section class="section">
    <div class="section-heading">
        <div class="section-number">05</div>
        <div>
            <h2>Application Documents</h2>
            <p>Upload your latest CV</p>
        </div>
    </div>
    <label class="upload-box" for="cv">
        <div class="upload-icon">↑</div>
        <div>
            <strong>Upload your CV</strong>
            <p>PDF, DOC or DOCX · Max 5 MB</p>
        </div>
        <input type="file" id="cv" accept=".pdf,.doc,.docx">
```

```
            </label>
            <small class="error" id="cvError"></small>
        </section>
        <div class="form-footer">
            <button type="reset" class="reset-btn">
                Clear Form
            </button>
            <button type="submit" class="submit-btn">
                Submit Application
                <span>→</span>
            </button>
        </div>
        <p class="footer-note">
            By submitting this form, you confirm that the information provided is accurate.
        </p>
    </form>
</main>
<script>
    const form = document.getElementById("jobForm");
    const name = document.getElementById("name");
    const age = document.getElementById("age");
    const password = document.getElementById("password");
    const confirmPassword = document.getElementById("confirmPassword");
    const securityQuestion = document.getElementById("securityQuestion");
    const securityAnswer = document.getElementById("securityAnswer");
    const email = document.getElementById("email");
    const phone = document.getElementById("phone");
```

```
const homePhone = document.getElementById("homePhone");
const cv = document.getElementById("cv");
function showError(input, errorId, message) {
const error = document.getElementById(errorId);
input.classList.add("input-error");
input.classList.remove("input-success");
error.textContent = message;
}
function showSuccess(input, errorId) {
const error = document.getElementById(errorId);
input.classList.remove("input-error");
input.classList.add("input-success");
error.textContent = "";
}
function clearValidation(input, errorId) {
const error = document.getElementById(errorId);
input.classList.remove("input-error", "input-success");

error.textContent = "";
}
function validateName() {
if (name.value.trim() === "") {
   showError(
   name,
   "nameError",
   "Please enter your name."
   );
```

```
    return false;
}
if (name.value.trim().length < 3) {
    showError(
    name,
    "nameError",
    "Name must contain at least 3 characters."
    );
    return false;
}
showSuccess(name, "nameError");
return true;
}
function validateAge() {
const value = Number(age.value);
if (age.value === "") {
    showError(
    age,
    "ageError",
    "Please enter your age."
    );
    return false;
}
if (value < 15 || value > 100) {
    showError(
    age,
    "ageError",
```

```
    "Age must be between 15 and 100."
    );
    return false;
}
showSuccess(age, "ageError");
return true;
}
function validatePassword() {
if (password.value === "") {
    showError(
    password,
    "passwordError",
    "Please enter a password."
    );
    return false;
}
if (password.value.length < 8) {
    showError(
    password,
    "passwordError",
    "Password must contain at least 8 characters."
    );
    return false;
}
showSuccess(password, "passwordError");
return true;
}
```

```
function validateConfirmPassword() {
if (confirmPassword.value === "") {
    showError(
    confirmPassword,
    "confirmPasswordError",
    "Please confirm your password."
    );
    return false;
}
if (confirmPassword.value !== password.value) {
    showError(
    confirmPassword,
    "confirmPasswordError",
    "Passwords do not match."
    );
    return false;
}
showSuccess(confirmPassword, "confirmPasswordError");
return true;
}
function validateSecurityQuestion() {
if (securityQuestion.value.trim() === "") {
    showError(
    securityQuestion,
    "securityQuestionError",
    "Please enter a security question."
    );
```

```
    return false;
}
if (securityQuestion.value.trim().length < 5) {
    showError(
    securityQuestion,
    "securityQuestionError",
    "Security question must contain at least 5 characters."
    );
    return false;
}
showSuccess(
    securityQuestion,
    "securityQuestionError"
);
return true;
}
function validateSecurityAnswer() {
if (securityAnswer.value.trim() === "") {
    showError(
    securityAnswer,
    "securityAnswerError",
    "Please enter a security answer."
    );
    return false;
}
if (securityAnswer.value.trim().length < 2) {
    showError(
```

```
    securityAnswer,
    "securityAnswerError",
    "Security answer must contain at least 2 characters."
    );
    return false;
}
showSuccess(
    securityAnswer,
    "securityAnswerError"
);
return true;
}
function validateEmail() {
const emailRegex =
    /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
if (email.value.trim() === "") {
    showError(
    email,
    "emailError",
    "Please enter your email address."
    );
    return false;
}
if (!emailRegex.test(email.value.trim())) {
    showError(
    email,
    "emailError",
```

```
    "Please enter a valid email address."
    );
    return false;
}
showSuccess(email, "emailError");
return true;
}
function validatePhone() {
const phoneRegex = /^[6-9][0-9]{9}$/;
if (phone.value.trim() === "") {
    showError(
    phone,
    "phoneError",
    "Please enter your phone number."
    );
    return false;
}
if (!phoneRegex.test(phone.value.trim())) {
    showError(
    phone,
    "phoneError",
    "Enter a valid 10-digit mobile number."
    );
    return false;
}
showSuccess(phone, "phoneError");
return true;
```

```
}
function validateHomePhone() {
if (homePhone.value.trim() === "") {
    clearValidation(homePhone, "homePhoneError");
    return true;
}
const phoneRegex = /^[0-9]{10}$/;
if (!phoneRegex.test(homePhone.value.trim())) {
    showError(
    homePhone,
    "homePhoneError",
    "Enter a valid 10-digit phone number."
    );
    return false;
}
showSuccess(homePhone, "homePhoneError");
return true;
}
function validateCV() {
const error = document.getElementById("cvError");
if (cv.files.length === 0) {
    error.textContent = "Please upload your CV.";
    cv.classList.add("input-error");
    return false;
}
const file = cv.files[0];
const allowedTypes = [
```

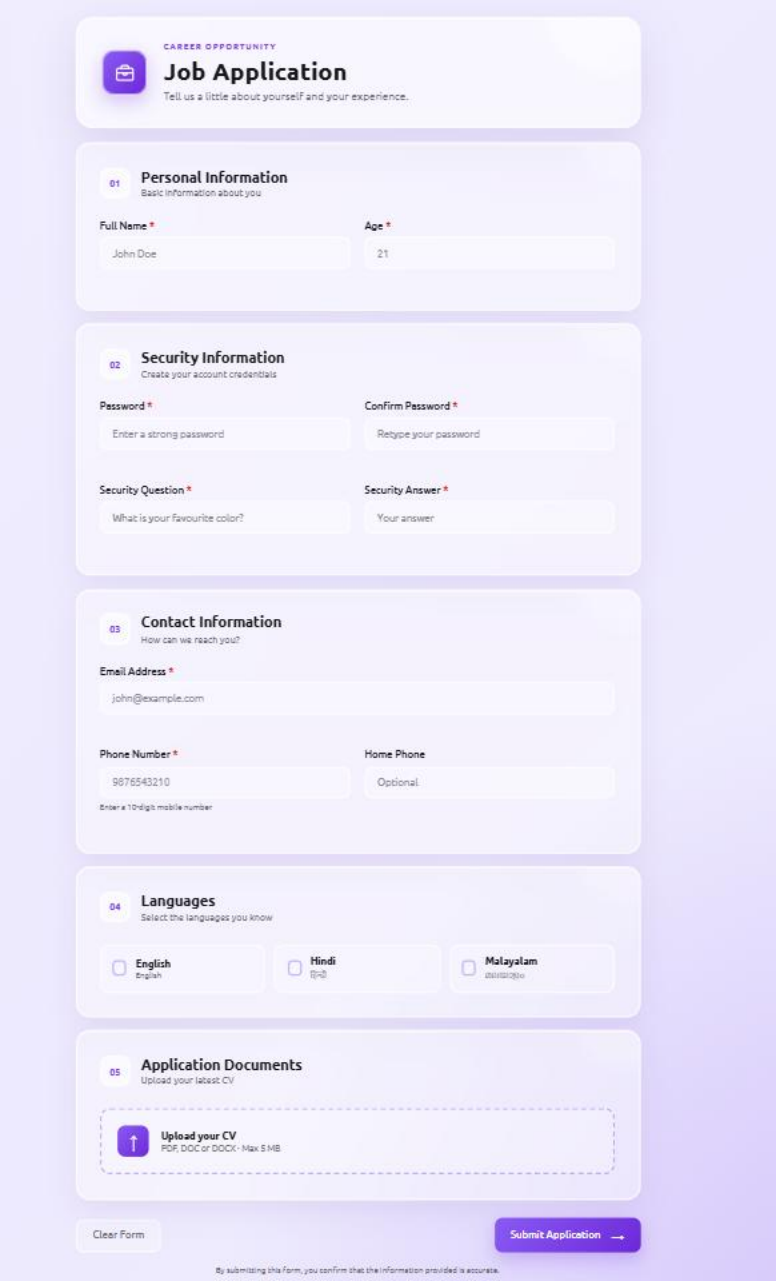
```
    "application/pdf",
    "application/msword",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document"
];
if (!allowedTypes.includes(file.type)) {
    error.textContent =
    "Only PDF, DOC, or DOCX files are allowed.";
    cv.classList.add("input-error");
    return false;
}
// Maximum file size = 5 MB
const maxSize = 5 * 1024 * 1024;
if (file.size > maxSize) {
    error.textContent =
    "File size must not exceed 5 MB.";
    cv.classList.add("input-error");
    return false;
}
error.textContent = "";
cv.classList.remove("input-error");
cv.classList.add("input-success");
return true;
}
name.addEventListener("input", validateName);
age.addEventListener("input", validateAge);
password.addEventListener("input", validatePassword);
securityQuestion.addEventListener("input",validateSecurityQuestion);
```

```
securityAnswer.addEventListener("input",validateSecurityAnswer);
email.addEventListener("input", validateEmail);
phone.addEventListener("input", validatePhone);

cv.addEventListener("change", validateCV);
form.addEventListener("submit", function (event) {
event.preventDefault();
const validName = validateName();
const validAge = validateAge();
const validPassword = validatePassword();
const validSecurityQuestion = validateSecurityQuestion();
const validSecurityAnswer = validateSecurityAnswer();
const validEmail = validateEmail();
const validHomePhone = validateHomePhone();
const validCV = validateCV();
const isValid =
    validName &&
    validAge &&
    validPassword &&
    validConfirmPassword &&
    validSecurityQuestion &&
    validSecurityAnswer &&
    validEmail &&
    validPhone &&
    validHomePhone &&
    validCV;
if (isValid) {
```

```
        alert("Application submitted successfully!");
    }
    });
    form.addEventListener("reset", function () {
    setTimeout(() => {
        const inputs = form.querySelectorAll("input");
        inputs.forEach(input => {
            "input-error",
            "input-success"
        );
        form.querySelectorAll(".error");
        errors.forEach(error => {
        error.textContent = "";
        });
    }, 0);
    });
  </script>
</body>
</html>
```

Output



Result

The Job Application Form was successfully designed and developed using HTML, CSS, and JavaScript. The form features a clean, responsive UI with glassmorphism design elements and real-time field validation to ensure accurate user input submission.

CSS Reference

Aim

Design and develop a webpage that presents different CSS reference topics. The webpage should contain a navigation section on the left and display the selected content in an iframe on the right.

Algorithm

1. Create the main HTML document and set the title to "CSS Reference".
2. Design a header containing the website logo and navigation links.
3. Create a hero section with a heading, subtitle, and description of the CSS reference website.
4. Add a scrolling text section displaying the available CSS topics.
5. Divide the main layout into a sidebar and a content area.
6. Create a sidebar with navigation links for Properties, Selectors, Functions, Animatable, Aural, Units, Colors, Default Values, and Browser Support.
7. Create properties.html to display CSS properties with descriptions and examples.
8. Create selectors.html to explain CSS selectors and their uses.
9. Create functions.html to display CSS functions and their purposes.
10. Create animatable.html to describe CSS properties that support animation.
11. Create aural.html to describe CSS properties related to speech and audio presentation.
12. Create units.html to explain CSS measurement units such as px, %, em, and rem.
13. Create colors.html to demonstrate CSS color formats and examples.
14. Create default-values.html to display default values of common CSS properties.
15. Create browser-support.html to describe browser compatibility for CSS features.
16. Display the selected reference page inside an iframe using target="contentFrame".
17. Apply CSS styles for fonts, colors, backgrounds, layouts, cards, tables, and hover effects.
18. Use JavaScript to highlight the selected sidebar link when clicked.
19. Add media queries to make the website responsive on mobile devices, tablets, and desktops.
20. Add a footer displaying the website information.
21. Save all HTML files in the appropriate folders and open index.html in a web browser to verify the navigation and page display.
22. Stop.

Source Code

index.html

<!DOCTYPE html>

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Reference</title>
  <link rel="icon" href="https://cdn.jsdelivr.net/gh/devicons/devicon/icons/css3/css3-original.svg">
  <style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Space+Grotesk:wght@500;600;700&display=swap');
    * {
      box-sizing: border-box;
    }
    html {
      scroll-behavior: smooth;
    }
    body {
      margin: 0;
      font-family: "Inter", Arial, sans-serif;
      background: radial-gradient(circle at 90% 10%, rgba(99, 102, 241, 0.12), transparent 30%), #f7f8fc;
      color: #171827;
    }
    .header {
      min-height: 70px;
      padding: 0 6%;
      display: flex;
```

```
    align-items: center;
    justify-content: space-between;
    background: white;
    border-bottom: 1px solid #e7e8ef;
    position: sticky;
    top: 0;
    z-index: 100;
}
.logo {
    display: flex;
    align-items: center;
    gap: 10px;
    font-family: "Space Grotesk", sans-serif;
    font-size: 20px;
    font-weight: 700;
}
.logo span {
    width: 38px;
    height: 38px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 10px;
    background: #6366f1;
    color: white;
}
.header-links {
```

```
    display: flex;
    gap: 30px;
    color: #73758a;
    font-size: 14px;
}
.hero {
    max-width: 1250px;
    margin: auto;
    padding: 70px 30px 50px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.hero-content {
    max-width: 720px;
}
.badge {
    display: inline-block;
    padding: 8px 13px;
    border-radius: 50px;
    background: #eeeeff;
    color: #5558d9;
    font-size: 11px;
    font-weight: 700;
    letter-spacing: 1px;
    margin-bottom: 20px;
}
```

```
.hero h1 {
    font-family: "Space Grotesk", sans-serif;
    font-size: clamp(42px, 6vw, 72px);
    line-height: 1.05;
    margin: 0;
    letter-spacing: -3px;
}
.hero h1 span {
    background: linear-gradient(90deg, #6366f1, #8b5cf6, #ec4899);
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
}
.hero p {
    max-width: 600px;
    margin-top: 25px;
    color: #707285;
    line-height: 1.8;
}
.hero-decoration {
    width: 200px;
    height: 200px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 40px;
    background: linear-gradient(135deg, #6366f1, #8b5cf6);
```

```
    color: white;
    font-family: monospace;
    font-size: 55px;
    transform: rotate(8deg);
    box-shadow: 0 30px 70px rgba(99, 102, 241, 0.3);
}
.ticker {
    background: #171827;
    color: white;
    padding: 13px 0;
    font-size: 13px;
}
.layout {
    max-width: 1250px;
    margin: 35px auto;
    padding: 0 20px;
    display: flex;
    gap: 25px;
}
.sidebar {
    width: 270px;
    flex-shrink: 0;
    padding: 20px;
    background: white;
    border: 1px solid #e5e6ee;
    border-radius: 20px;
    box-shadow: 0 10px 40px rgba(20, 20, 40, 0.05);
```

```
    height: fit-content;
}
.menu-title {
    display: flex;
    justify-content: space-between;
    margin-bottom: 15px;
    padding: 5px 5px 15px;
    border-bottom: 1px solid #eee;
}
.menu-title span {
    font-size: 11px;
    font-weight: 800;
    letter-spacing: 1.5px;
    color: #8a8b9a;
}
.menu-title small {
    color: #aaa;
}
.menu-item {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 12px 13px;
    margin: 4px 0;
    border-radius: 11px;
    text-decoration: none;
    color: #5e6072;
```

```
    font-size: 13px;
    transition: 0.25s;
}
.menu-item .icon {
    width: 25px;
    height: 25px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 7px;
    background: #f1f1f8;
}
.menu-item:hover {
    background: #f4f3ff;
    color: #5758d9;
    transform: translateX(3px);
}
.menu-item.active {
    background: #6366f1;
    color: white;
}
.menu-item.active .icon {
    background: rgba(255, 255, 255, 0.2);
    color: white;
}
.content {
    flex: 1;
```

```
    min-width: 0;
}
iframe {
    display: block;
    width: 100%;
    min-height: 700px;
    border: none;
    border-radius: 20px;
    background: white;
    box-shadow: 0 10px 40px rgba(20, 20, 40, 0.05);
}
footer {
    text-align: center;
    padding: 25px;
    color: #8b8c99;
    font-size: 12px;
}
@media (max-width: 900px) {
    .hero-decoration {
        display: none;
    }
    .layout {
        flex-direction: column;
    }
    .sidebar {
        width: 100%;
    }
```

```
    .menu-item {
        display: inline-flex;
        width: calc(50% - 5px);
    }
}
@media (max-width: 600px) {
    .header {
        padding: 0 20px;
    }
    .header-links {
        display: none;
    }
    .hero {
        padding: 50px 20px 35px;
    }
    .hero h1 {
        font-size: 45px;
        letter-spacing: -2px;
    }
    .layout {
        margin-top: 20px;
        padding: 0 10px;
    }
    .sidebar {
        padding: 15px;
    }
    .menu-item {
```

```
            width: 100%;
        }
    }
  </style>
</head>
<body>
  <header class="header">
    <div class="logo">
      <span>&lt;/&gt;</span>
      CSS Reference
    </div>

    <div class="header-links">
      <span>Learn</span>
      <span>Explore</span>
      <span>Reference</span>
    </div>
  </header>
  <section class="hero">
    <div class="hero-content">
      <span class="badge">✦ CSS DOCUMENTATION</span>
      <h1>
        Master CSS.<br>
        <span>One reference at a time.</span>
      </h1>
      <p>
        Explore CSS properties, selectors, functions, units,
```

```
            colors and browser compatibility in one place.
        </p>
    </div>
    <div class="hero-decoration">&lt;/&gt;</div>
</section>
<div class="ticker">
    <marquee>
        ✦ CSS Properties   
        ✦ CSS Selectors   
        ✦ CSS Functions   
        ✦ CSS Units   
        ✦ CSS Colors   
        ✦ Browser Support
    </marquee>
</div>
<main class="layout">
    <aside class="sidebar">
        <div class="menu-title">
            <span>REFERENCE</span>
            <small>9 topics</small>
        </div>
        <a href="pages/properties.html" target="contentFrame"
           class="menu-item active" onclick="setActive(this)">
            <span class="icon">◈</span> Properties
        </a>
        <a href="pages/selectors.html" target="contentFrame"
```

```
   class="menu-item" onclick="setActive(this)">
    <span class="icon">⌘</span> Selectors
</a>
<a href="pages/functions.html" target="contentFrame"
   class="menu-item" onclick="setActive(this)">
    <span class="icon">ƒ</span> Functions
</a>
<a href="pages/animatable.html" target="contentFrame"
   class="menu-item" onclick="setActive(this)">
    <span class="icon">✦</span> Animatable
</a>
<a href="pages/aural.html" target="contentFrame"
   class="menu-item" onclick="setActive(this)">
    <span class="icon">◉</span> Aural
</a>
<a href="pages/units.html" target="contentFrame"
   class="menu-item" onclick="setActive(this)">
    <span class="icon">#</span> Units
</a>
<a href="pages/colors.html" target="contentFrame"
   class="menu-item" onclick="setActive(this)">
    <span class="icon">●</span> Colors
</a>
<a href="pages/default-values.html" target="contentFrame"
   class="menu-item" onclick="setActive(this)">
    <span class="icon">=</span> Default Values
</a>
```

```
            <a href="pages/browser-support.html" target="contentFrame"
               class="menu-item" onclick="setActive(this)">
                <span class="icon">□</span> Browser Support
            </a>
        </aside>
        <section class="content">
            <iframe src="pages/properties.html"
                    name="contentFrame"
                    title="CSS Reference Content">
            </iframe>
        </section>
    </main>
    <footer>
        <p>CSS Reference • Built with HTML &amp;amp; CSS</p>
    </footer>
    <script>
        function setActive(element) {
            document.querySelectorAll(".menu-item").forEach(function(item) {
                item.classList.remove("active");
            });
            element.classList.add("active");
        }
    </script>
</body>
</html>
```

animatable.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>CSS Animatable</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="reference-page">
        <div class="reference-header">
            <span class="reference-label">CSS REFERENCE / 04</span>
            <h1>CSS Animatable</h1>
            <p>Animatable properties can smoothly change their values when used with transitions or animations.</p>
        </div>
        <div class="cards">
            <div class="card">
                <div class="card-icon">↔</div>
                <h3>Width</h3>
                <p>Width can transition smoothly.</p>
                <br>
                <code>width: 300px;</code>
            </div>
            <div class="card">
                <div class="card-icon">◐</div>
                <h3>Opacity</h3>
```

```
            <p>Controls transparency.</p>
            <br>
            <code>opacity: .5;</code>
        </div>
        <div class="card">
            <div class="card-icon">↻</div>
            <h3>Transform</h3>
            <p>Allows smooth transformations.</p>
            <br>
            <code>transform: rotate(45deg);</code>
        </div>
    </div>
    <h2>Animatable Properties</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>Property</th><th>Animatable</th><th>Example</th></tr>
            <tr><td><code>color</code></td><td>Yes</td><td><code>color:
red;</code></td></tr>
            <tr><td><code>background-color</code></td><td>Yes</td><td><code>background-
color: blue;</code></td></tr>
            <tr><td><code>width</code></td><td>Yes</td><td><code>width:
300px;</code></td></tr>
            <tr><td><code>height</code></td><td>Yes</td><td><code>height:
200px;</code></td></tr>
            <tr><td><code>opacity</code></td><td>Yes</td><td><code>opacity:
0.5;</code></td></tr>
            <tr><td><code>transform</code></td><td>Yes</td><td><code>transform:
scale(1.2);</code></td></tr>
```

```
            <tr><td><code>margin</code></td><td>Yes</td><td><code>margin: 20px;</code></td></tr>
        </table>
    </div>
  </div>
</body>
</html>
```

aural.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Aural</title>
  <link rel="stylesheet" href="../style.css">
</head>
<body>
  <div class="reference-page">
    <div class="reference-header">
      <span class="reference-label">CSS REFERENCE / 05</span>
      <h1>CSS Aural</h1>
      <p>Aural CSS was designed to control how web content is presented through speech synthesizers and screen readers.</p>
    </div>
    <div class="cards">
      <div class="card">
        <div class="card-icon">🔊</div>
        <h3>Volume</h3>
```

```
            <p>Controls speech volume.</p>
        </div>
        <div class="card">
            <div class="card-icon">◉</div>
            <h3>Voice</h3>
            <p>Controls voice characteristics.</p>
        </div>
        <div class="card">
            <div class="card-icon">◷</div>
            <h3>Pause</h3>
            <p>Controls pauses during speech.</p>
        </div>
    </div>
    <h2>Aural Properties</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>Property</th><th>Description</th><th>Example</th></tr>
            <tr><td><code>volume</code></td><td>Controls speech volume.</td><td><code>volume: loud;</code></td></tr>
            <tr><td><code>voice-family</code></td><td>Specifies the voice family.</td><td><code>voice-family: male;</code></td></tr>
            <tr><td><code>speech-rate</code></td><td>Controls speech speed.</td><td><code>speech-rate: fast;</code></td></tr>
            <tr><td><code>pause-before</code></td><td>Adds a pause before speaking.</td><td><code>pause-before: 2s;</code></td></tr>
            <tr><td><code>pause-after</code></td><td>Adds a pause after speaking.</td><td><code>pause-after: 2s;</code></td></tr>
        </table>
    </div>
```

```
        <p><strong>Note:</strong> Many traditional aural CSS properties are now obsolete or have limited browser support.</p>
    </div>
</body>
</html>
```

browser-support.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>CSS Browser Support</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="reference-page">
        <div class="reference-header">
            <span class="reference-label">CSS REFERENCE / 09</span>
            <h1>Browser Support</h1>
            <p>Browser compatibility determines whether a particular CSS feature works correctly across different browsers.</p>
        </div>
        <div class="cards">
            <div class="card">
                <div class="card-icon">C</div>
                <h3>Chrome</h3>
                <p>Modern CSS features are widely supported.</p>
            </div>
```

```
        <div class="card">
            <div class="card-icon">F</div>
            <h3>Firefox</h3>
            <p>Supports modern CSS standards.</p>
        </div>
        <div class="card">
            <div class="card-icon">S</div>
            <h3>Safari</h3>
            <p>Supports most modern CSS features.</p>
        </div>
    </div>
    <h2>Browser Compatibility</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>CSS
Feature</th><th>Chrome</th><th>Firefox</th><th>Edge</th><th>Safari</th></tr>
            <tr><td>CSS Grid</td><td>✓ Supported</td><td>✓ Supported</td><td>✓
Supported</td><td>✓ Supported</td></tr>
            <tr><td>Flexbox</td><td>✓ Supported</td><td>✓ Supported</td><td>✓
Supported</td><td>✓ Supported</td></tr>
            <tr><td>CSS Animations</td><td>✓ Supported</td><td>✓ Supported</td><td>✓
Supported</td><td>✓ Supported</td></tr>
            <tr><td>CSS Variables</td><td>✓ Supported</td><td>✓ Supported</td><td>✓
Supported</td><td>✓ Supported</td></tr>
            <tr><td>Media Queries</td><td>✓ Supported</td><td>✓ Supported</td><td>✓
Supported</td><td>✓ Supported</td></tr>
        </table>
    </div>
```

```
        <h2>Why Compatibility Matters</h2>
        <p>A website may be viewed using different browsers and devices. Checking browser support helps developers provide a consistent experience for users.</p>
    </div>
</body>
</html>
```

colors.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>CSS Colors</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="reference-page">
        <div class="reference-header">
            <span class="reference-label">CSS REFERENCE / 07</span>
            <h1>CSS Colors</h1>
            <p>CSS provides several ways to define colors, including names, hexadecimal, RGB and HSL values.</p>
        </div>
        <div class="cards">
            <div class="card">
                <div class="card-icon">#</div>
                <h3>HEX</h3>
                <p>Uses hexadecimal notation.</p>
```

```
            <br>
            <code>#6366f1</code>
        </div>
        <div class="card">
            <div class="card-icon">RGB</div>
            <h3>RGB</h3>
            <p>Uses red, green and blue values.</p>
            <br>
            <code>rgb(99,102,241)</code>
        </div>
        <div class="card">
            <div class="card-icon">HSL</div>
            <h3>HSL</h3>
            <p>Uses hue, saturation and lightness.</p>
            <br>
            <code>hsl(239,84%,67%)</code>
        </div>
    </div>
    <h2>Color Formats</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>Format</th><th>Description</th><th>Example</th></tr>
            <tr><td>Color Name</td><td>Uses predefined color
names.</td><td><code>red</code></td></tr>
            <tr><td>HEX</td><td>Uses hexadecimal
values.</td><td><code>#ff0000</code></td></tr>
            <tr><td>RGB</td><td>Uses red, green and blue
values.</td><td><code>rgb(255,0,0)</code></td></tr>
```

```
            <tr><td>RGBA</td><td>RGB with transparency.</td><td><code>rgba(255,0,0,.5)</code></td></tr>
            <tr><td>HSL</td><td>Uses hue, saturation and lightness.</td><td><code>hsl(0,100%,50%)</code></td></tr>
            <tr><td>HSLA</td><td>HSL with transparency.</td><td><code>hsla(0,100%,50%,.5)</code></td></tr>
        </table>
    </div>
  </div>
</body>
</html>
```

default-values.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Default Values</title>
  <link rel="stylesheet" href="../style.css">
</head>
<body>
  <div class="reference-page">
    <div class="reference-header">
      <span class="reference-label">CSS REFERENCE / 08</span>
      <h1>CSS Default Values</h1>
      <p>CSS properties have default values that browsers use when a value is not explicitly specified.</p>
    </div>
```

```
<div class="cards">
    <div class="card">
        <div class="card-icon">1</div>
        <h3>Opacity</h3>
        <p>Default opacity is completely visible.</p>
        <br>
        <code>opacity: 1</code>
    </div>
    <div class="card">
        <div class="card-icon">S</div>
        <h3>Position</h3>
        <p>Elements are positioned statically by default.</p>
        <br>
        <code>position: static</code>
    </div>
    <div class="card">
        <div class="card-icon">V</div>
        <h3>Visibility</h3>
        <p>Elements are visible by default.</p>
        <br>
        <code>visibility: visible</code>
    </div>
</div>
<h2>Default Values</h2>
<div class="table-wrapper">
    <table>
        <tr><th>Property</th><th>Default Value</th><th>Description</th></tr>
```

```
            <tr><td><code>position</code></td><td>static</td><td>Default positioning method.</td></tr>

            <tr><td><code>opacity</code></td><td>1</td><td>Element is completely opaque.</td></tr>

            <tr><td><code>visibility</code></td><td>visible</td><td>Element is visible.</td></tr>

            <tr><td><code>overflow</code></td><td>visible</td><td>Overflowing content remains visible.</td></tr>

            <tr><td><code>font-style</code></td><td>normal</td><td>Text is not italic.</td></tr>

            <tr><td><code>font-weight</code></td><td>normal</td><td>Normal font thickness.</td></tr>

          </table>

      </div>

   </div>

</body>

</html>
```

functions.html

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <title>CSS Functions</title>

   <link rel="stylesheet" href="../style.css">

</head>

<body>

   <div class="reference-page">

      <div class="reference-header">
```

```
        <span class="reference-label">CSS REFERENCE / 03</span>
        <h1>CSS Functions</h1>
        <p>CSS functions provide special values and allow calculations to be performed directly inside CSS.</p>
    </div>
    <div class="cards">
        <div class="card">
            <div class="card-icon">∑</div>
            <h3>calc()</h3>
            <p>Performs mathematical calculations.</p>
            <br>
            <code>calc(100% - 20px)</code>
        </div>
        <div class="card">
            <div class="card-icon">RGB</div>
            <h3>rgb()</h3>
            <p>Creates colors using RGB values.</p>
            <br>
            <code>rgb(99,102,241)</code>
        </div>
        <div class="card">
            <div class="card-icon">V</div>
            <h3>var()</h3>
            <p>Uses CSS custom properties.</p>
            <br>
            <code>var(--color)</code>
        </div>
```

```
    </div>
    <h2>Common Functions</h2>
    <div class="table-wrapper">
      <table>
        <tr><th>Function</th><th>Purpose</th><th>Example</th></tr>
        <tr><td><code>calc()</code></td><td>Performs calculations.</td><td><code>width: calc(100% - 20px);</code></td></tr>
        <tr><td><code>rgb()</code></td><td>Creates an RGB color.</td><td><code>rgb(255, 0, 0)</code></td></tr>
        <tr><td><code>rgba()</code></td><td>Creates an RGB color with transparency.</td><td><code>rgba(255,0,0,.5)</code></td></tr>
        <tr><td><code>hsl()</code></td><td>Creates an HSL color.</td><td><code>hsl(240,100%,50%)</code></td></tr>
        <tr><td><code>var()</code></td><td>Uses a CSS variable.</td><td><code>var(--main-color)</code></td></tr>
        <tr><td><code>min()</code></td><td>Returns the smallest value.</td><td><code>min(90%,500px)</code></td></tr>
        <tr><td><code>max()</code></td><td>Returns the largest value.</td><td><code>max(50%,300px)</code></td></tr>
      </table>
    </div>
  </div>
</body>
</html>
```

properties.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>CSS Properties</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="reference-page">
        <div class="reference-header">
            <span class="reference-label">CSS REFERENCE / 01</span>
            <h1>CSS Properties</h1>
            <p>CSS properties define how HTML elements look, behave and are positioned on a webpage.</p>
        </div>
        <div class="cards">
            <div class="card">
                <div class="card-icon">A</div>
                <h3>Color</h3>
                <p>Controls the color of text.</p>
                <br>
                <code>color: #6366f1;</code>
            </div>
            <div class="card">
                <div class="card-icon">Bg</div>
                <h3>Background</h3>
                <p>Controls the background of an element.</p>
                <br>
                <code>background: white;</code>
            </div>
```

```
        <div class="card">
            <div class="card-icon">↔</div>
            <h3>Spacing</h3>
            <p>Controls margin and padding.</p>
            <br>
            <code>padding: 20px;</code>
        </div>
    </div>
    <h2>Common Properties</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>Property</th><th>Description</th><th>Example</th></tr>
            <tr><td><code>color</code></td><td>Sets the text color.</td><td><code>color: red;</code></td></tr>
            <tr><td><code>background-color</code></td><td>Sets the background color.</td><td><code>background-color: blue;</code></td></tr>
            <tr><td><code>font-size</code></td><td>Sets the text size.</td><td><code>font-size: 20px;</code></td></tr>
            <tr><td><code>margin</code></td><td>Sets outside spacing.</td><td><code>margin: 20px;</code></td></tr>
            <tr><td><code>padding</code></td><td>Sets inside spacing.</td><td><code>padding: 20px;</code></td></tr>
            <tr><td><code>border</code></td><td>Creates a border around an element.</td><td><code>border: 1px solid;</code></td></tr>
            <tr><td><code>width</code></td><td>Controls element width.</td><td><code>width: 300px;</code></td></tr>
            <tr><td><code>height</code></td><td>Controls element height.</td><td><code>height: 200px;</code></td></tr>
        </table>
    </div>
```

```
    </div>
</body>
</html>
```

selectors.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>CSS Selectors</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="reference-page">
        <div class="reference-header">
            <span class="reference-label">CSS REFERENCE / 02</span>
            <h1>CSS Selectors</h1>
            <p>Selectors identify the HTML elements that CSS rules should be applied to.</p>
        </div>
        <div class="cards">
            <div class="card">
                <div class="card-icon">*</div>
                <h3>Universal</h3>
                <p>Selects every element.</p>
                <br>
                <code>* { }</code>
            </div>
```

```
        <div class="card">
            <div class="card-icon">.</div>
            <h3>Class</h3>
            <p>Selects elements by class.</p>
            <br>
            <code>.box { }</code>
        </div>
        <div class="card">
            <div class="card-icon">#</div>
            <h3>ID</h3>
            <p>Selects an element by ID.</p>
            <br>
            <code>#header { }</code>
        </div>
    </div>
    <h2>Selector Types</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>Selector</th><th>Name</th><th>Example</th></tr>
            <tr><td><code>*</code></td><td>Universal</td><td><code>* { margin: 0;
}</code></td></tr>
            <tr><td><code>p</code></td><td>Element</td><td><code>p { color: red;
}</code></td></tr>
            <tr><td><code>.box</code></td><td>Class</td><td><code>.box { padding: 10px;
}</code></td></tr>
            <tr><td><code>#header</code></td><td>ID</td><td><code>#header { height: 80px;
}</code></td></tr>
            <tr><td><code>div p</code></td><td>Descendant</td><td><code>div p { color:
blue; }</code></td></tr>
```

```
            <tr><td><code>div &gt; p</code></td><td>Child</td><td><code>div &gt; p { color: green; }</code></td></tr>
            <tr><td><code>a:hover</code></td><td>Pseudo-class</td><td><code>a:hover { color: red; }</code></td></tr>
        </table>
    </div>
  </div>
</body>
</html>
```

units.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Units</title>
  <link rel="stylesheet" href="../style.css">
</head>
<body>
  <div class="reference-page">
    <div class="reference-header">
      <span class="reference-label">CSS REFERENCE / 06</span>
      <h1>CSS Units</h1>
      <p>CSS units define measurements for properties such as width, height, spacing and font size.</p>
    </div>
    <div class="cards">
      <div class="card">
```

```
            <div class="card-icon">px</div>
            <h3>Pixels</h3>
            <p>A commonly used fixed unit.</p>
            <br>
            <code>20px</code>
        </div>
        <div class="card">
            <div class="card-icon">%</div>
            <h3>Percentage</h3>
            <p>Relative to a parent or reference value.</p>
            <br>
            <code>50%</code>
        </div>
        <div class="card">
            <div class="card-icon">rem</div>
            <h3>REM</h3>
            <p>Relative to the root font size.</p>
            <br>
            <code>2rem</code>
        </div>
    </div>
    <h2>Common CSS Units</h2>
    <div class="table-wrapper">
        <table>
            <tr><th>Unit</th><th>Type</th><th>Description</th><th>Example</th></tr>

<tr><td><code>px</code></td><td>Absolute</td><td>Pixels.</td><td><code>20px</code></td></tr>
```

<tr><td><code>%</code></td><td>Relative</td><td>Percentage value.</td><td><code>50%</code></td></tr>

<tr><td><code>em</code></td><td>Relative</td><td>Relative to element font size.</td><td><code>2em</code></td></tr>

<tr><td><code>rem</code></td><td>Relative</td><td>Relative to root font size.</td><td><code>2rem</code></td></tr>

<tr><td><code>vw</code></td><td>Relative</td><td>Relative to viewport width.</td><td><code>50vw</code></td></tr>

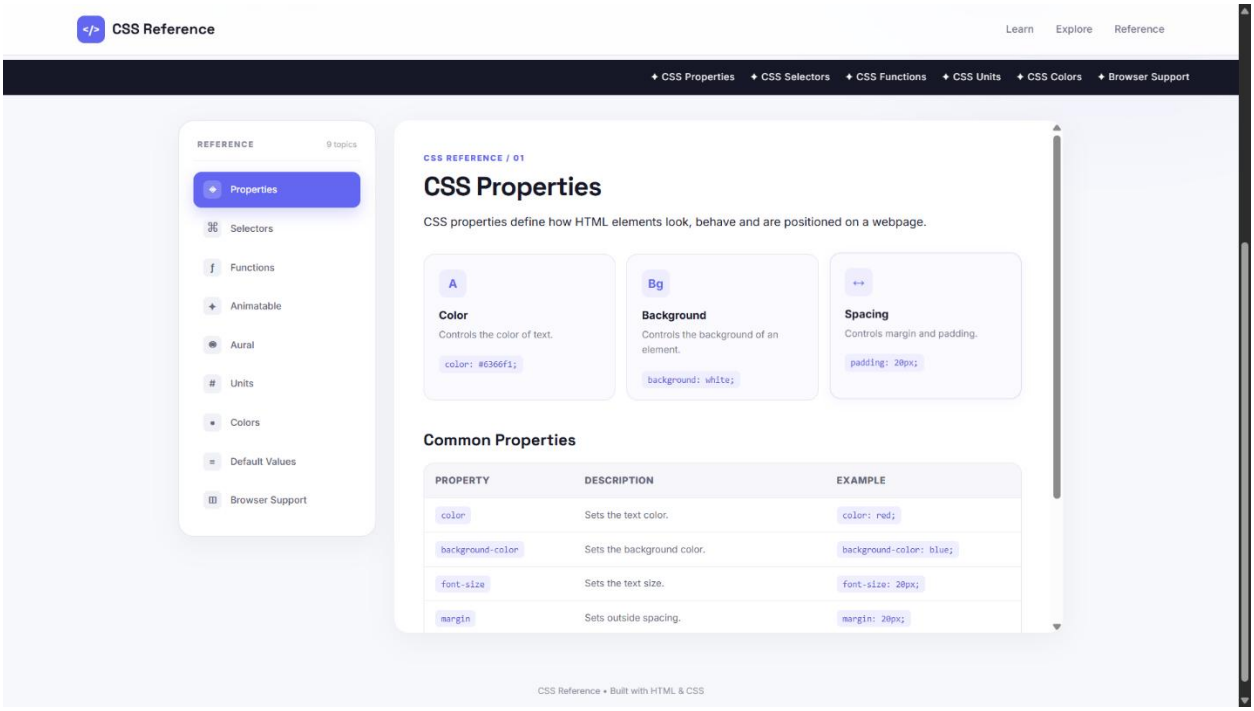
<tr><td><code>vh</code></td><td>Relative</td><td>Relative to viewport height.</td><td><code>50vh</code></td></tr>

</table>

</body>

</html>

Output



Results

The CSS Reference webpage was successfully created and tested. The navigation menu displays the selected CSS topic in the content frame as intended.

String and Math Functions

Aim

Design and develop a webpage that accepts user input and demonstrates different String functions and Math functions using JavaScript.

Algorithm

1. Start the program.
2. Create the HTML structure with a header, navigation menu, input form, output console, and footer.
3. Use CSS to style the webpage with a responsive layout, colors, buttons, and input fields.
4. Define JavaScript arrays containing string functions and mathematical operations with their default values.
5. Display the available functions in the navigation menu.
6. When a function is selected, dynamically generate the required input fields and update the function details.
7. When the Run Operation button is clicked, validate the inputs and execute the selected function.
8. Display the result or an error message in the output console.
9. Implement Math Mode and String Mode switching, along with Copy Result and Reset buttons.
10. Use responsive CSS media queries to adjust the layout for smaller screens.
11. Test the functions, input validation, mode switching, and output display.
12. Stop the program.

Source Code

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>FunctionLab | JavaScript Playground</title>

    <link rel="icon" href="https://cdn-icons-png.flaticon.com/512/5968/5968292.png">

    <style>

        * {
```

```
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
:root {
    --primary: #7c3aed;
    --light: #ede9fe;
    --bg: #f7f7fb;
    --text: #24213a;
    --muted: #77758b;
    --border: #e7e4ef;
}
body {
    font-family: "Segoe UI", Arial, sans-serif;
    background: var(--bg);
    color: var(--text);
    min-height: 100vh;
}
body.math {
    --primary: #0891b2;
    --light: #cffafe;
    --bg: #f2f9fc;
}
header {
    background: white;
    padding: 20px 6%;
    display: flex;
```

```
    justify-content: space-between;
    align-items: center;
    border-bottom: 1px solid var(--border);
}
header h2 {
    font-size: 23px;
}
button {
    cursor: pointer;
    font: inherit;
    border: 0;
    border-radius: 8px;
    padding: 11px 15px;
}
#mode, #copy, #reset {
    background: var(--light);
    color: var(--primary);
    font-weight: 600;
}
main {
    max-width: 1200px;
    margin: auto;
    padding: 35px 24px;
}
.intro {
    margin-bottom: 28px;
}
```

```
.intro p {
    color: var(--primary);
    font-size: 12px;
    font-weight: bold;
    letter-spacing: 2px;
    margin-bottom: 10px;
}
.intro h1 {
    font-size: clamp(25px, 4vw, 34px);
    margin-bottom: 10px;
}
.intro span, #opDesc, #message, .info p {
    color: var(--muted);
    font-size: 14px;
    line-height: 1.7;
}
.layout {
    display: grid;
    grid-template-columns: 210px minmax(0, 1.3fr) minmax(220px, .8fr);
    gap: 20px;
    align-items: start;
}
nav, .panel, .output {
    background: white;
    border: 1px solid var(--border);
    border-radius: 14px;
    padding: 22px;
```

```
}
nav h3 {
    font-size: 11px;
    letter-spacing: 1px;
    color: var(--muted);
    margin-bottom: 15px;
}
nav ul {
    list-style: none;
    display: grid;
    gap: 5px;
}
.nav-item {
    width: 100%;
    text-align: left;
    background: transparent;
    color: var(--muted);
    font-size: 13px;
}
.nav-item:hover, .nav-item.active {
    background: var(--light);
    color: var(--primary);
    font-weight: bold;
}
.panel h2 {
    font-size: 21px;
    margin-bottom: 8px;
```

```
}
#opDesc {
    margin-bottom: 25px;
}
.field {
    margin-bottom: 17px;
}
.field label {
    display: block;
    font-size: 13px;
    font-weight: 600;
    margin-bottom: 8px;
}
.field input, .field select {
    width: 100%;
    padding: 12px;
    border: 1px solid var(--border);
    border-radius: 8px;
    background: var(--bg);
    color: var(--text);
    outline: none;
}
.field input:focus, .field select:focus {
    border-color: var(--primary);
}
.primary {
    width: 100%;
```

```
    background: var(--primary);
    color: white;
    font-weight: bold;
    margin-top: 5px;
}
.output h3 {
    font-size: 16px;
    margin-bottom: 18px;
}
.result {
    background: var(--light);
    padding: 20px 15px;
    border-radius: 10px;
    min-height: 130px;
    overflow-wrap: anywhere;
}
.result small {
    color: var(--primary);
    font-weight: bold;
    font-size: 10px;
    letter-spacing: 1px;
}
.result h2 {
    margin-top: 15px;
    font-size: 24px;
    white-space: pre-wrap;
}
```

```
#message {
  margin: 15px 0;
}
.actions {
  display: flex;
  gap: 8px;
}
.actions button {
  flex: 1;
  font-size: 12px;
}
.info {
  border-top: 1px solid var(--border);
  margin-top: 22px;
  padding-top: 18px;
}
.info h4 {
  margin-bottom: 10px;
}
.info code {
  display: inline-block;
  background: var(--bg);
  color: var(--primary);
  padding: 8px;
  border-radius: 6px;
  margin-top: 12px;
  font-size: 12px;
```

```
    overflow-wrap: anywhere;
}
footer {
    text-align: center;
    color: var(--muted);
    font-size: 12px;
    padding: 20px;
}
@media (max-width: 950px) {
    .layout {
        grid-template-columns: 1fr 1fr;
    }
    nav {
        grid-column: 1 / -1;
    }
    nav ul {
        grid-template-columns: repeat(3, 1fr);
    }
}
@media (max-width: 600px) {
    header {
        padding: 16px;
    }
    header h2 {
        font-size: 19px;
    }
    #mode {
```

```
            font-size: 12px;
            padding: 10px;
        }
        main {
            padding: 25px 15px;
        }
        .layout {
            grid-template-columns: 1fr;
        }
        nav ul {
            grid-template-columns: 1fr 1fr;
        }
        nav, .panel, .output {
            padding: 18px;
        }
    }
  </style>
</head>
<body>
  <header>
    <h2>ƒ FunctionLab</h2>
    <button id="mode">Math Mode</button>
  </header>
  <main>
    <section class="intro">
      <p>INTERACTIVE LEARNING SPACE</p>
      <h1 id="heading">Explore String Functions</h1>
```

```
    <span>Experiment with JavaScript functions and see instant results.</span>
</section>
<section class="layout">
    <nav>
        <h3 id="navTitle">STRING FUNCTIONS</h3>
        <ul id="navList"></ul>
    </nav>
    <section class="panel">
        <h2 id="opTitle">String Length</h2>
        <p id="opDesc">Find the number of characters in a string.</p>
        <form id="form">
            <div id="fields"></div>
            <button type="submit" class="primary">Run Operation</button>
        </form>
    </section>
    <aside class="output">
        <h3>Output Console</h3>
        <div class="result">
            <small id="status">YOUR RESULT</small>
            <h2 id="result">Your result will appear here.</h2>
        </div>
        <p id="message">Select a function and enter your values.</p>
        <div class="actions">
            <button type="button" id="copy">Copy Result</button>
            <button type="button" id="reset">Reset</button>
        </div>
        <div class="info">
```

```
            <h4>About this function</h4>
            <p id="info">Explore JavaScript's built-in functions.</p>
            <code id="example">text.length</code>
          </div>
        </aside>
      </section>
    </main>
    <footer>Built with HTML, CSS and JavaScript · FunctionLab</footer>
    <script>
      const $ = id => document.getElementById(id);
      const T = value => ["text", value];
      const N = value => ["number", value];
      const strings = [
        ["Length", {text: T("Hello JavaScript")}, v => v.text.length],
        ["First Occurrence", {text: T("Hello JavaScript"), word: T("JavaScript")}, v =>
v.text.indexOf(v.word)],
        ["Last Occurrence", {text: T("Hello JavaScript Hello"), word: T("Hello")}, v =>
v.text.lastIndexOf(v.word)],
        ["Search", {text: T("Learn JavaScript"), word: T("JavaScript")}, v =>
v.text.search(v.word)],
        ["Slice", {text: T("JavaScript"), start: N(0), end: N(4)}, v => v.text.slice(v.start, v.end)],
        ["Substring", {text: T("JavaScript"), start: N(4), end: N(10)}, v => v.text.substring(v.start,
v.end)],
        ["Substr", {text: T("JavaScript"), start: N(4), length: N(6)}, v => v.text.substr(v.start,
v.length)],
        ["Replace", {text: T("Hello World"), old: T("World"), new: T("JavaScript")}, v =>
v.text.replace(v.old, v.new)],
        ["Uppercase", {text: T("Hello JavaScript")}, v => v.text.toUpperCase()],
        ["Lowercase", {text: T("HELLO JAVASCRIPT")}, v => v.text.toLowerCase()]
```

```
    ];
    const maths = [
        ["Power", {base: N(2), exponent: N(3)}, v => Math.pow(v.base, v.exponent)],
        ["Square Root", {number: N(25)}, v => Math.sqrt(v.number)],
        ["Round", {number: N(4.6)}, v => Math.round(v.number)],
        ["Ceiling", {number: N(4.2)}, v => Math.ceil(v.number)],
        ["Floor", {number: N(4.8)}, v => Math.floor(v.number)],
        ["Absolute Value", {number: N(-25.5)}, v => Math.abs(v.number)],
        ["Sine", {angle: N(30)}, v => Math.sin(v.angle * Math.PI / 180)],
        ["Cosine", {angle: N(60)}, v => Math.cos(v.angle * Math.PI / 180)],
        ["Minimum", {numbers: T("10, 25, -5, 4.5")}, v =>
Math.min(...parseNumbers(v.numbers))],
        ["Maximum", {numbers: T("10, 25, -5, 4.5")}, v =>
Math.max(...parseNumbers(v.numbers))],
        ["Random Number", {min: N(1), max: N(100)}, v => {
            if (!Number.isInteger(v.min) || !Number.isInteger(v.max) || v.max < v.min)
                throw Error("Enter valid whole-number limits.");
            return Math.floor(Math.random() * (v.max - v.min + 1)) + v.min;
        }]
    ];
    let mode = strings, selected = strings[0], result = "";
    function parseNumbers(text) {
        const values = text.split(",").map(x => x.trim());
        if (values.some(x => x === "" || !Number.isFinite(Number(x))))
            throw Error("Enter valid comma-separated numbers.");
        return values.map(Number);
    }
    function showMenu() {
```

```
        $("navTitle").textContent = mode === strings ? "STRING FUNCTIONS" : "MATH FUNCTIONS";
        $("heading").textContent = mode === strings ? "Explore String Functions" : "Explore Math Functions";
        $("navList").innerHTML = mode.map((op, i) =>
            `<li><button class="nav-item ${op === selected ? "active" : ""}" data-id="${i}">${op[0]}</button></li>`
        ).join("");
        $("navList").querySelectorAll("button").forEach(btn => {
            btn.onclick = () => {
                selected = mode[Number(btn.dataset.id)];
                showMenu();
                showFields();
            };
        });
    }
    function showFields() {
        $("opTitle").textContent = selected[0];
        $("opDesc").textContent = "Enter the values and run the operation.";
        $("example").textContent = selected[2].toString();
        $("info").textContent = "JavaScript function: " + selected[0];
        $("fields").innerHTML = Object.entries(selected[1]).map(([key, data]) => `
            <div class="field">
                <label for="${key}">${key}</label>
                <input id="${key}" type="${data[0]}" step="any" value="${data[1]}" placeholder="Enter ${key}">
            </div>
        `).join("");
```

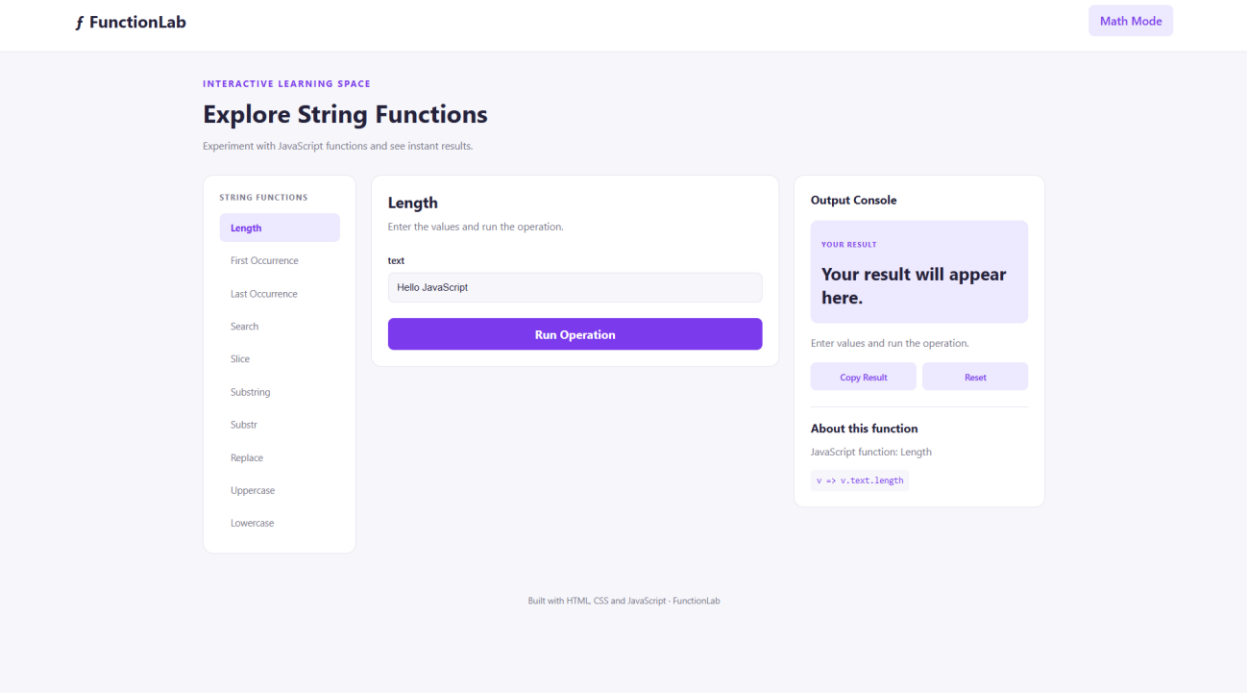
```
    $("result").textContent = "Your result will appear here.";
    $("status").textContent = "YOUR RESULT";
    $("message").textContent = "Enter values and run the operation.";
    result = "";
}
$("form").onsubmit = e => {
    e.preventDefault();
    try {
        const values = {};
        Object.entries(selected[1]).forEach(([key, data]) => {
            const input = $(key);
            if (input.value.trim() === "")
                throw Error("Please fill in all fields.");
            values[key] = data[0] === "number" ? Number(input.value) : input.value;
            if (data[0] === "number" && !Number.isFinite(values[key]))
                throw Error("Enter valid numeric values.");
        });

        const output = selected[2](values);
        if (typeof output === "number" && !Number.isFinite(output))
            throw Error("Result is undefined for these values.");
        result = String(output);
        $("result").textContent = result;
        $("status").textContent = "SUCCESS";
        $("message").textContent = "Operation completed successfully.";
    } catch (error) {
        result = "";
```

```
        $("result").textContent = error.message;
        $("status").textContent = "INPUT ERROR";
        $("message").textContent = "Check your input and try again.";
    }
};
$("mode").onclick = () => {
    mode = mode === strings ? maths : strings;
    selected = mode[0];
    document.body.classList.toggle("math", mode === maths);
    $("mode").textContent = mode === strings ? "Math Mode" : "String Mode";
    showMenu();
    showFields();
};
$("copy").onclick = async () => {
    if (!result) {
        $("message").textContent = "Run an operation first.";
        return;
    }
    try {
        await navigator.clipboard.writeText(result);
        $("message").textContent = "Result copied successfully.";
    } catch {
        $("message").textContent = "Clipboard unavailable. Copy the result manually.";
    }
};
$("reset").onclick = showFields;
showMenu();
```

```
        showFields();
    </script>
</body>
</html>
```

Output



Result

The FunctionLab webpage was successfully created using HTML, CSS, and JavaScript. It allows users to explore string functions and mathematical operations interactively, view results instantly, switch modes, copy results, and reset inputs.

Calendar

Aim

Design and develop a webpage that accepts a year and month as input and dynamically generates the corresponding monthly calendar.

Algorithm

1. Start the program.
2. Create input fields for the year and month, along with Generate Calendar and Reset buttons.
3. Apply CSS to style the calendar and arrange dates in a seven-column grid.
4. Get the year and month entered by the user.
5. Validate the inputs and display an alert if they are invalid.
6. Calculate the first day of the month and the total number of days using JavaScript's Date object.
7. Display the month and year as the calendar title.
8. Generate the weekday headings and arrange the dates in the correct positions.
9. Implement the Reset button to clear the inputs and calendar.
10. Test the webpage and stop the program.

Source Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Monthly Calendar</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      background: #f4f4f4;

      display: flex;

      justify-content: center;
```

```
    padding: 30px;
}
.container {
    background: white;
    padding: 20px;
    border-radius: 10px;
    width: 350px;
    text-align: center;
}
input, select, button {
    padding: 8px;
    margin: 5px 0;
}
button {
    cursor: pointer;
    border: none;
    background: #333;
    color: white;
    border-radius: 5px;
}
#calendar {
    display: grid;
    grid-template-columns: repeat(7, 1fr);
    gap: 5px;
}
#calendar div {
    padding: 8px 0;
```

```
            background: #eee;
            border-radius: 4px;
        }
        #calendar .day {
            background: #333;
            color: white;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>Monthly Calendar</h2>
        <input type="number" id="year" placeholder="Year" min="1">
        <select id="month">
            <option value="">Select Month</option>
            <option value="0">January</option>
            <option value="1">February</option>
            <option value="2">March</option>
            <option value="3">April</option>
            <option value="4">May</option>
            <option value="5">June</option>
            <option value="6">July</option>
            <option value="7">August</option>
            <option value="8">September</option>
            <option value="9">October</option>
            <option value="10">November</option>
            <option value="11">December</option>
```

```
    </select>
    <button onclick="generate()">Generate Calendar</button>
    <button onclick="resetCalendar()">Reset</button>
    <h3 id="title"></h3>
    <div id="calendar"></div>
</div>
<script>
    function generate() {
        let year = Number(document.getElementById("year").value);
        let month = document.getElementById("month").value;
        if (!year || year < 1 || month === "") {
            alert("Enter a valid year and select a month!");
            return;
        }
        month = Number(month);
        let firstDay = new Date(year, month, 1).getDay();
        let days = new Date(year, month + 1, 0).getDate();
        let calendar = document.getElementById("calendar");
        calendar.innerHTML = "";
        document.getElementById("title").textContent =
            new Date(year, month).toLocaleString("en", {
                month: "long", year: "numeric"
            });
        ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"].forEach(day => {
            calendar.innerHTML += `<div class="day">${day}</div>`;
        });
        for (let i = 0; i < firstDay; i++) {
```

```
            calendar.innerHTML += "<div></div>";
        }
        for (let i = 1; i <= days; i++) {
            calendar.innerHTML += `<div>${i}</div>`;
        }
    }
    function resetCalendar() {
        document.getElementById("year").value = "";
        document.getElementById("month").value = "";
        document.getElementById("title").textContent = "";
        document.getElementById("calendar").innerHTML = "";
    }
  </script>
</body>
</html>
```

Output



Result

The Monthly Calendar webpage was successfully created and tested. It dynamically generates the selected month's calendar with correctly arranged dates and provides Generate and Reset functionality.

Form Validation

Aim

Design and develop a form that collects user information and validates the entered data before submission.

Algorithm

1. Start the program.
2. Create a registration form with fields for name, email, mobile number, username, and password.
3. Apply CSS to style and position the form.
4. Retrieve the values entered by the user when the form is submitted.
5. Validate the name to allow only letters and spaces.
6. Validate the email by checking the positions of @ and ..
7. Check that the mobile number contains exactly 10 digits.
8. Ensure the username is not empty.
9. Verify that the password contains at least 8 characters, including uppercase and lowercase letters, a digit, and a special character.
10. Display an error message if any validation fails; otherwise, display a registration success message.
11. Stop the program.

Source Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Registration Form</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      background: #f4f4f4;

      display: flex;
```

```
            justify-content: center;
            padding: 30px;
        }
        form {
            background: white;
            padding: 20px;
            width: 300px;
            border-radius: 8px;
        }
        input, button {
            width: 100%;
            padding: 10px;
            margin: 8px 0;
            box-sizing: border-box;
        }
        button {
            background: #333;
            color: white;
            border: none;
            cursor: pointer;
        }
        button:hover {
            background: #555;
        }
    </style>
</head>
<body>
```

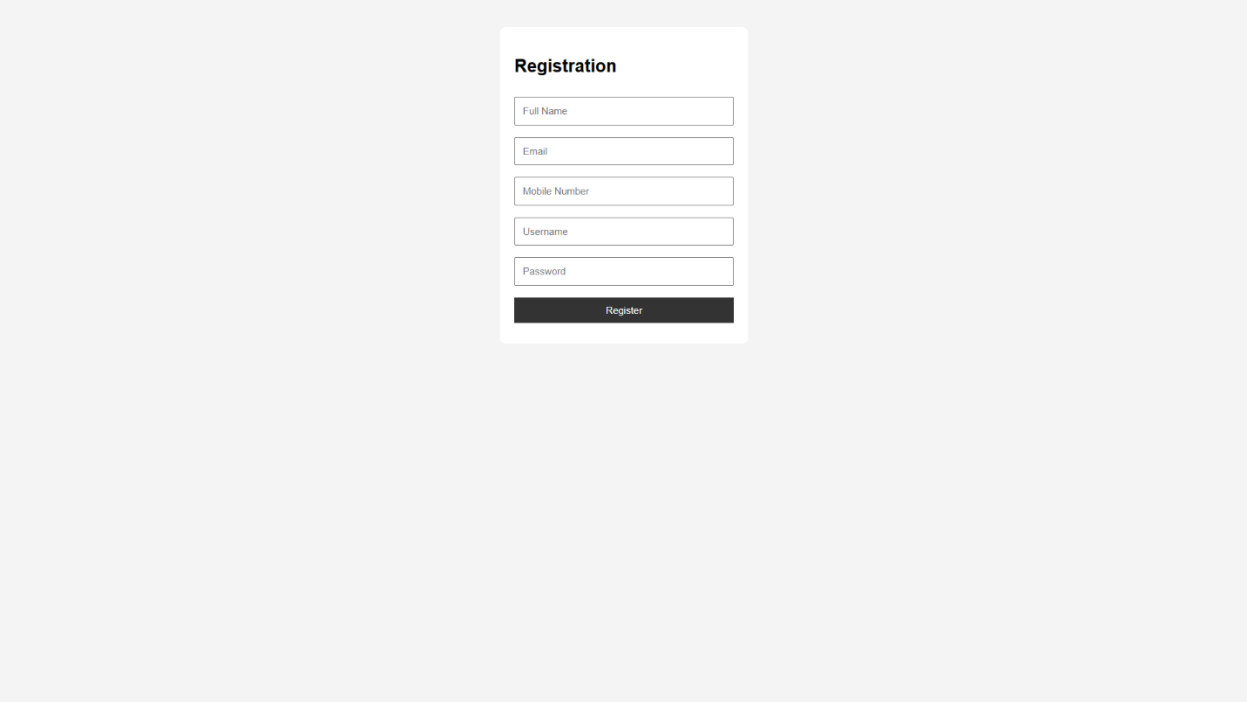
```
<form onsubmit="return validateForm()">
    <h2>Registration</h2>
    <input type="text" id="name" placeholder="Full Name">
    <input type="text" id="email" placeholder="Email">
    <input type="text" id="mobile" placeholder="Mobile Number">
    <input type="text" id="username" placeholder="Username">
    <input type="password" id="password" placeholder="Password">
    <button type="submit">Register</button>
</form>
<script>
    function validateForm() {
        let name = document.getElementById("name").value.trim();
        let email = document.getElementById("email").value.trim();
        let mobile = document.getElementById("mobile").value.trim();
        let username = document.getElementById("username").value.trim();
        let password = document.getElementById("password").value;
        for (let ch of name) {
            if (!((ch >= "A" && ch <= "Z") || (ch >= "a" && ch <= "z") || ch === " ")) {
                alert("Name must contain only letters and spaces.");
                return false;
            }
        }
        if (name === "") {
            alert("Enter a valid name.");
            return false;
        }
        let at = email.indexOf("@");
```

```
let dot = email.lastIndexOf(".");
if (at < 1 || dot < at + 2 || dot === email.length - 1) {
    alert("Enter a valid email address.");
    return false;
}
if (mobile.length !== 10) {
    alert("Mobile number must contain exactly 10 digits.");
    return false;
}
for (let ch of mobile) {
    if (ch < "0" || ch > "9") {
        alert("Mobile number must contain only digits.");
        return false;
    }

if (username === "") {
    alert("Username cannot be empty.");
    return false;
}
let upper = false, lower = false, digit = false, special = false;
for (let ch of password) {
    if (ch >= "A" && ch <= "Z") upper = true;
    else if (ch >= "a" && ch <= "z") lower = true;
    else if (ch >= "0" && ch <= "9") digit = true;
    else special = true;
}
if (password.length < 8 || !upper || !lower || !digit || !special) {
```

```
            alert("Password needs 8 characters, uppercase, lowercase, digit and special character.");
            return false;
        }
        alert("Registration successful!");
        return true;
    }
  </script>
</body>
</html>
```

Output



Result

The registration form was successfully created using HTML, CSS, and JavaScript. It validates user inputs and displays appropriate error messages or a success message based on the entered details.

Dynamic Background Colour Change

Aim

Design and develop a webpage with a button that changes the background color whenever it is clicked.

Algorithm

1. Start the program.
2. Create a webpage with a heading, description, button, and color display.
3. Use CSS to style and center the content.
4. Select the button and color display using JavaScript.
5. Generate a random hexadecimal color code using six random characters.
6. Add a click event listener to the button.
7. When clicked, change the webpage background to the generated color.
8. Display the new color code on the webpage.
9. Repeat the process whenever the button is clicked.
10. Stop the program.

Source Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Random Background Changer</title>

  <style>

    * {

      margin: 0;

      padding: 0;

      box-sizing: border-box;

    }

    body {

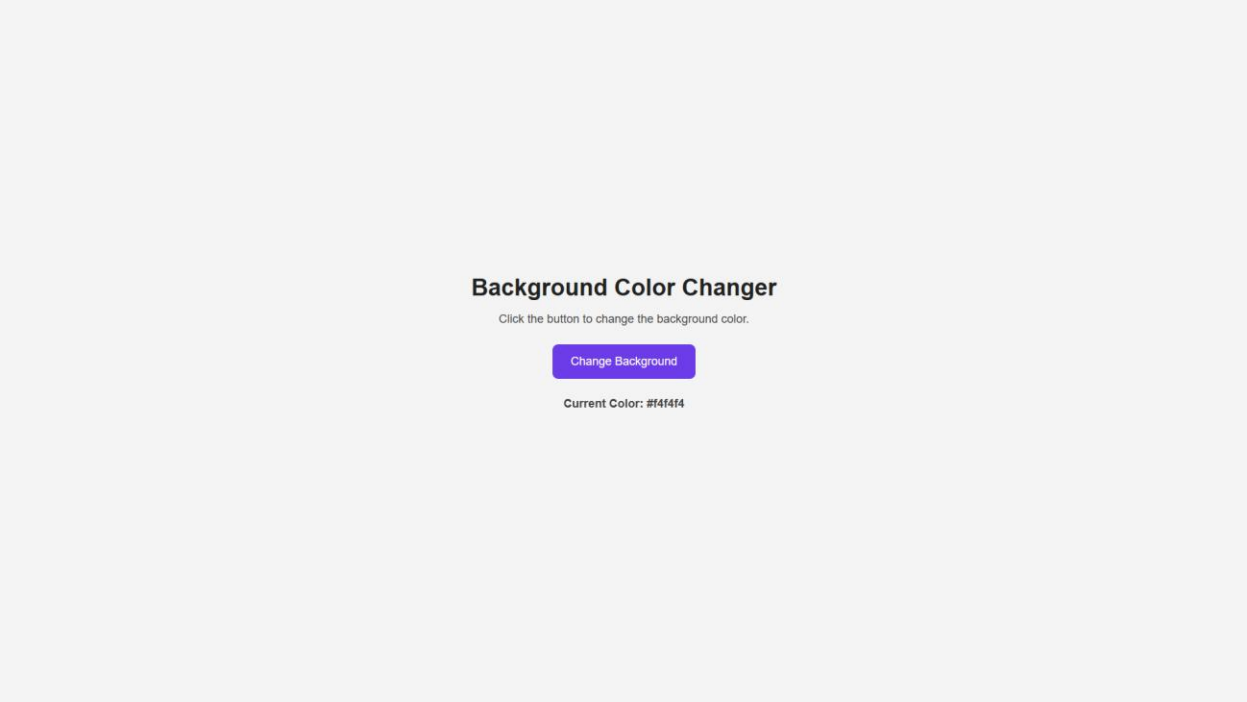
      font-family: Arial, sans-serif;
```

```
    background-color: #f4f4f4;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    text-align: center;
    transition: background-color 0.4s ease;
}
.container {
    padding: 30px;
}
h1 {
    color: #222;
    margin-bottom: 15px;
}
p {
    color: #444;
    margin-bottom: 25px;
}
button {
    padding: 14px 25px;
    background-color: #6c3ce9;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 16px;
    cursor: pointer;
```

```
            transition: transform 0.2s, background-color 0.2s;
        }
        button:hover {
            background-color: #5125c9;
            transform: scale(1.05);
        }
        button:active {
            transform: scale(0.97);
        }
        #colorCode {
            margin-top: 25px;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Background Color Changer</h1>
        <p>Click the button to change the background color.</p>
        <button id="changeBtn">Change Background</button>
        <p id="colorCode">Current Color: #f4f4f4</p>
    </div>
    <script>
        const button = document.getElementById("changeBtn");
        const colorCode = document.getElementById("colorCode");
        function generateRandomColor() {
            const letters = "0123456789ABCDEF";
```

```
        let color = "#";
        for (let i = 0; i < 6; i++) {
            const randomIndex = Math.floor(Math.random() * 16);
            color += letters[randomIndex];
        }
        return color;
    }
    function change() {
        const randomColor = generateRandomColor();
        document.body.style.backgroundColor = randomColor;
        colorCode.textContent = "Current Color: " + randomColor;
    }
    button.addEventListener("click", change);
  </script>
</body>
</html>
```

Output



Result

The Random Background Changer webpage was successfully created and tested. Clicking the button changes the background to a random color and displays its hexadecimal code.

Mouse Over Image Change

Aim

Design and develop a webpage that changes the displayed image and text when the user moves the mouse over and away from the content area.

Algorithm

1. Start the program.
2. Create a webpage with a heading, image, and description.
3. Use CSS to center the content and style the image.
4. Select the content area, image, and headings using JavaScript.
5. Add a mouseover event to display the alternate image and heading.
6. Add a mouseout event to restore the original image and heading.
7. Test the webpage by moving the mouse over and away from the content area.
8. Stop the program.

Source Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Mouse Hover Event</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      text-align: center;

      margin: 0;

      width: 100%;

      height: 100vh;

      display: flex;

      justify-content: center;
```

```
            align-items: center;
        }
        .container {
            max-width: 80%;
            text-align: center;
        }
        h1, p {
            color: black;
        }
        img {
            display: block;
            width: 100%;
            max-width: 500px;
            height: 300px;
            margin: 20px auto;
            border-radius: 10px;
        }
    </style>
</head>
<body>
    <div class="container" id="content">
        <h1 id="heading">Welcome to My Website!</h1>
        <h1 id="alternateHeading" hidden>Hovered! Image Changed.</h1>
        <img id="image" src="./assets/img/original-image.svg" alt="Original image">
        <p>Move your mouse over the image to see the change.</p>
    </div>
    <script>
```

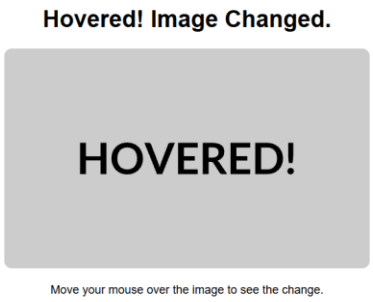
```
        const content = document.getElementById("content");
        const image = document.getElementById("image");
        const heading = document.getElementById("heading");
        const alternateHeading = document.getElementById("alternateHeading");
        const originalImage = "./assets/img/original-image.svg";
        const hoverImage = "./assets/img/hovered.svg";

        content.addEventListener("mouseover", function() {
            image.src = hoverImage;
            heading.hidden = true;
            alternateHeading.hidden = false;
        });

        content.addEventListener("mouseout", function() {
            image.src = originalImage;
            heading.hidden = false;
            alternateHeading.hidden = true;
        });
    </script>
</body>
</html>
```

Output





Result

The webpage was successfully created and tested. Moving the mouse over the content area changes the image and heading, while moving it away restores the original content.